IF4031 Arsitektur Aplikasi Terdistribusi

Milestone 1

RMB Distributed



Kelompok 11: RMB

Nathaniel Jonathan Rusli	13523013
Maheswara Bayu Kaindra	13523015
Buege Mahara Putra	13523037

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

2026

Daftar Isi

1. Deskripsi Umum Sistem.....6
1.1. Gambaran Umum Sistem..... 6
1.2. Lingkup *Milestone* 1.....6
1.3. Asumsi.....6
2. Arsitektur Sistem.....8
2.1. Diagram Arsitektur.....8
2.2. Pemilihan Teknologi.....9
2.3. Petunjuk Menjalankan Sistem.....10
3. *Problem Set*.....11
3.1. *Problem* 1: Arsitektur & Interoperabilitas Data..... 11
3.1.1. Rancangan Solusi..... 11
3.1.2. Alasan Keputusan.....14
3.1.3. Batasan..... 14
3.1.4. Bukti Penyelesaian..... 15
3.1.5. Alternatif Solusi..... 17
3.2. *Problem* 2: *Concurrency, Availability, & Graceful Degradation*.....19
3.2.1. Rancangan Solusi..... 19
3.2.2. Alasan Keputusan.....21
3.2.3. Batasan..... 22
3.2.4. Bukti Penyelesaian..... 22
3.2.5. Pembahasan: Konkurensi, Pencegahan Kegagalan, dan Load Testing..... 26
3.3. *Problem* 3: *Service-to-Service Authentication & Trust*.....28
3.3.1. Rancangan Solusi..... 28
3.3.2. Alasan Keputusan.....30
3.3.3. Batasan..... 32
3.3.4. Bukti Penyelesaian..... 32
3.4. *Problem* 4: Arsitektur *Microservice* & *Flexible Storage*..... 35
3.4.1. Rancangan Solusi..... 35
3.4.2. Alasan Keputusan.....37
3.4.3. Batasan..... 38
3.4.4. Bukti Penyelesaian..... 39
3.4.5. Pembahasan: Penentuan Batas *Service* dan Alternatif Penyimpanan..... 44
3.5. *Problem* 5: *Fan-out Event Distribution/Pub-sub*..... 46
3.5.1. Rancangan Solusi..... 46
3.5.2. Alasan Keputusan.....48
3.5.3. Batasan..... 49
3.5.4. Bukti Penyelesaian..... 49
Tabel Kontribusi.....55
Lampiran.....56

1. Deskripsi Umum Sistem

1.1. Gambaran Umum Sistem

Sistem ini merupakan *gateway* koordinasi BNPB yang menjembatani dua lembaga otonom, yaitu BMKG yang memantau aktivitas seismik serta potensi tsunami, dan PVMBG yang memantau aktivitas gunung api. Sistem mengambil data dengan *polling* secara berkala ke BMKG dan PVMBG, melakukan *mapping* ke dalam satu format kanonik `HazardEvent`, lalu menyimpannya pada Canonical Store. Data kanonik tersebut disajikan kepada tiga jenis *downstream client* (Media, Tim Lapangan, dan BNPB Pusat) dengan hak akses yang berbeda, dan didistribusikan melalui *service* internal dengan *message broker*.

1.2. Lingkup *Milestone* 1

Lingkup pengerjaan dibatasi sesuai dengan spesifikasi *Milestone* 1 sebagai berikut.

1. Terdapat dua *source*, BMKG dan PVMBG, yang disimulasikan sebagai *mock service* pasif yang hanya merespons permintaan *polling*.
2. BNPB berperan sebagai *node* koordinasi yang terdiri atas *Aggregator, Canonical Store, Client-facing API, Auth Service, message broker*, dan dua *event consumer*, ditambah satu *consumer* ketiga yang digunakan untuk Problem 5.
3. Seluruh *client* bersifat *read-only*, sehingga jalur pelaporan dari lapangan ke BNPB berada di luar lingkup pengerjaan.

1.3. Asumsi

Perancangan dan implementasi sistem didasarkan pada asumsi berikut.

1. *Source* hanya melakukan perubahan schema yang bersifat aditif, yaitu penambahan *field*. Penghapusan, penggantian nama, dan perubahan tipe *field* wajib berada di luar lingkup.
2. Pasangan `source` dan `source_ref_id` bersifat unik dan tidak berubah, sehingga dapat digunakan sebagai *idempotency key* saat penyimpanan. Untuk menjamin hal ini, *mock service* menyertakan identitas proses pada setiap `event_id` dan `report_id`, sehingga *restart mock service* tidak menghasilkan identitas yang bertabrakan dengan data yang sudah tersimpan.
3. Jam pada *source* dan *Aggregator* tersinkronisasi, dan *timestamp* setiap *record* (`occurred_at`, `reported_at`, dan `issued_at`) tidak lebih awal dari waktu *record* tersebut tersedia pada *source*. Asumsi ini diperlukan karena *Aggregator* menggunakan

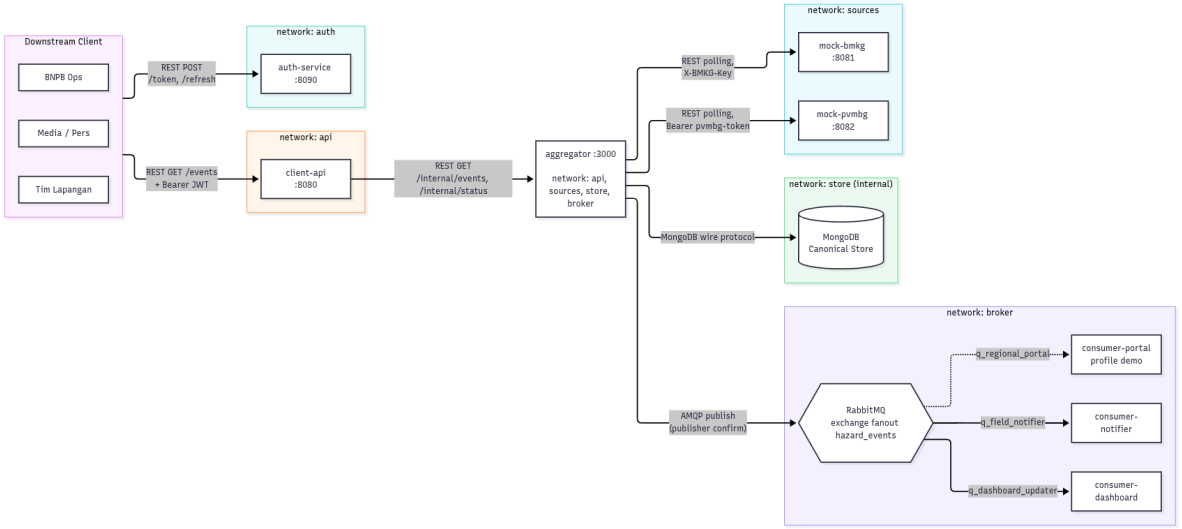
waktu *polling* terakhir yang berhasil sebagai nilai *parameter* `since` pada *polling* berikutnya.

4. Setiap `TsunamiWarning` selalu merujuk ke `SeismicEvent` yang sudah ada, tetapi dapat terbit setelah kejadian tersebut diterima BNPB.
5. Apabila satu kejadian memiliki beberapa `TsunamiWarning`, peringatan dengan tingkat ancaman tertinggi yang berlaku, dan di antara peringatan yang setara dipilih yang terbit paling akhir. Peringatan yang lebih lemah dan datang belakangan tidak menurunkan tingkat ancaman.
6. Nama dan koordinat gunung api berasal dari tabel referensi statis milik BNPB yang disematkan pada *Aggregator*, karena laporan PVMBG hanya menyertakan `volcano_id`.
7. Data dari suatu *source* dianggap *stale* apabila *polling* terakhir ke *source* tersebut gagal, apabila *source* belum pernah berhasil di-*polling*, atau apabila keberhasilan terakhirnya telah melampaui 10 detik.
8. Setiap jenis *downstream client* direpresentasikan oleh satu identitas *client* dengan secret tersendiri, sedangkan pengguna perorangan di dalam organisasi *client* tidak dimodelkan.
9. *Auth Service* dijalankan sebagai satu *instance*, sehingga penyimpanan *refresh token* dan penghitung percobaan *login* di memori dianggap memadai.
10. Seluruh *container* berjalan pada satu *host* Docker dengan *network* internal yang dipercaya, sehingga komunikasi antar-*container* tanpa enkripsi dianggap memadai.
11. *Canonical Store* merupakan sumber kebenaran, sehingga kehilangan *event* pada jalur distribusi dalam kondisi tertentu masih dapat diterima karena data tetap dapat dibaca melalui *Aggregator*.
12. Pengiriman *event* ke *consumer* bersifat *at-least-once*, sehingga consumer dapat menerima pesan yang sama lebih dari sekali. Duplikasi tersebut disaring oleh *consumer* berdasarkan pasangan `hazard_id` dan `severity`.
13. Status *toggle schema* dan *outage* pada *mock service* PVMBG tidak persisten dan kembali ke nilai bawaan ketika *mock service* dinyalakan ulang.

2. Arsitektur Sistem

2.1. Diagram Arsitektur

Gambar 2.1.1 memperlihatkan seluruh *service*, protokol komunikasi antar-*service*, kepemilikan penyimpanan, alur *event*, serta batas *container* dan *network*. Garis putus-putus menunjukkan *consumer* ketiga yang hanya dijalankan untuk demonstrasi *Problem* 5.



Gambar 2.1.1 Diagram Arsitektur Sistem

Setiap komponen pada **Gambar 2.1.1** dijalankan sebagai *container* Docker tersendiri, dan setiap garis menunjukkan arah panggilan beserta protokol yang digunakan. Secara garis besar, sistem memiliki tiga alur utama, yaitu alur *ingest,* alur baca, dan alur distribusi *event*.

Pada alur *ingest*, *Aggregator* menjalankan dua *poller*, satu untuk setiap source, yang masing-masing berjalan secara berkala (*default* setiap 3 detik). Data yang diterima di-*mapping* ke `HazardEvent`, disimpan ke MongoDB dengan operasi *upsert*, dan setiap `HazardEvent` yang benar-benar baru di-*publish* ke RabbitMQ. Apabila `TsunamiWarning` terbit setelah kejadian seismiknya tersimpan, *Aggregator* memperbarui tingkat ancaman kejadian tersebut dan mem-*publish* ulang perubahannya. *Aggregator* merupakan satu-satunya komponen yang berkomunikasi dengan *source* dan satu-satunya *writer Canonical Store*.

Pada alur baca, *client* terlebih dahulu memperoleh *access token* dari *Auth Service*, kemudian memanggil *Client-facing* API dengan menyertakan *token* tersebut. *Client-facing* API kemudian memverifikasi *token* secara lokal menggunakan *public key*, lalu membaca data melalui API internal *Aggregator*. *Client-facing* API tidak memanggil *mock service* secara langsung, sehingga waktu respons ke *client* tidak dipengaruhi oleh faktor kelambatan *source*.

Pada alur distribusi *event*, *Aggregator* melakukan *publish* `HazardEvent` ke sebuah *exchange* bertipe *fanout* melalui *confirm channel*. Setiap *consumer* memiliki *queue* tersendiri yang terikat ke *exchange* tersebut, sehingga *Aggregator* tidak perlu mengetahui keberadaan *consumer* manapun.

Setiap penyimpanan dimiliki oleh tepat satu *service*, dan *service* lain yang membutuhkan datanya harus memintanya melalui API milik *service* tersebut. *Canonical Store* dimiliki oleh *Aggregator* dan ditempatkan pada *network* `store` yang bersifat internal serta hanya dapat diakses oleh *Aggregator*. Penyimpanan *refresh token* dimiliki oleh *Auth Service*, sedangkan *seed* data, status *outage*, dan status versi *schema* dimiliki oleh masing-masing *mock service*.

Pemisahan *network* dirancang mengikuti batas kepercayaan antarkomponen, seperti ditunjukkan pada **Tabel 2.1.1**.

Tabel 2.1.1 Pembagian *Network* Docker

Network	**Anggota**	**Keterangan**
`sources`	`aggregator`, `mock-bmkg`, `mock-pvmbg`	Jalur *polling* ke *source*
`store`	`aggregator`, `mongo`	Bersifat internal
`broker`	`aggregator`, `rabbitmq`, seluruh *consumer*	Jalur distribusi *event*
`api`	`aggregator`, `client-api`	Jalur baca dari *Client-facing* API
`auth`	`auth-service`	-

2.2. Pemilihan Teknologi

Pemilihan teknologi didasarkan pada karakteristik beban sistem yang bersifat I/O-*bound*, kebutuhan penyimpanan dengan *schema* yang dapat berubah, serta kebutuhan distribusi *event* kepada banyak penerima. **Tabel 2.2.1** memuat teknologi yang digunakan pada setiap komponen beserta alasan pemilihannya.

Tabel 2.2.1 Teknologi Terpilih dan Alasan Pemilihannya

Komponen	**Teknologi**	**Alasan Pemilihan**
Seluruh service	Node.js 22, TypeScript, Express	Model *event loop* yang *non-blocking* sesuai untuk beban sistem I/O-*bound*.

Komponen	Teknologi	Alasan Pemilihan
Canonical Store	MongoDB 7	Memungkinkan penyimpanan dokumen tanpa *schema* tetap, sehingga *field* baru dapat disimpan tanpa *migration*.
Message broker	RabbitMQ 4 (AMQP 0-9-1)	Menyediakan *exchange fanout* untuk pola *pub-sub*, *durable queue* per *consumer, publisher confirm*, dan *acknowledgement* manual.
Token client	JWT RS256 dengan library `jose`	Skema asimetris yang memisahkan *issuer* (pemegang *private key*) dan *verifier* (pemegang *public key*).
Logging	pino	Menghasilkan *structured log* berformat JSON yang memuat *correlation ID* dan *latency* setiap *outgoing call*.
Load testing	k6	Mendukung skenario paralel, *threshold* p95 dan *error rate,* dan alur autentikasi langsung dalam *script*.
Orkestrasi	Docker Compose v2	Seluruh sistem dapat dinyalakan dengan satu perintah, dilengkapi *health check*, urutan *startup* berbasis *health status*, isolasi *network*, dan *profile* untuk *service* demonstrasi.

2.3. Petunjuk Menjalankan Sistem

Langkah lengkap untuk menjalankan sistem dari kondisi awal, daftar variabel konfigurasi, serta perintah untuk memicu setiap skenario pengujian tercantum pada file `README.md` pada repositori.

Salin `.env.example` menjadi `.env`, bangkitkan seluruh kredensial dengan menjalankan *script* `scripts/gen-secrets.sh`, lalu jalankan *command* di bawah ini dan tunggu hingga seluruh *container* berstatus *healthy*.

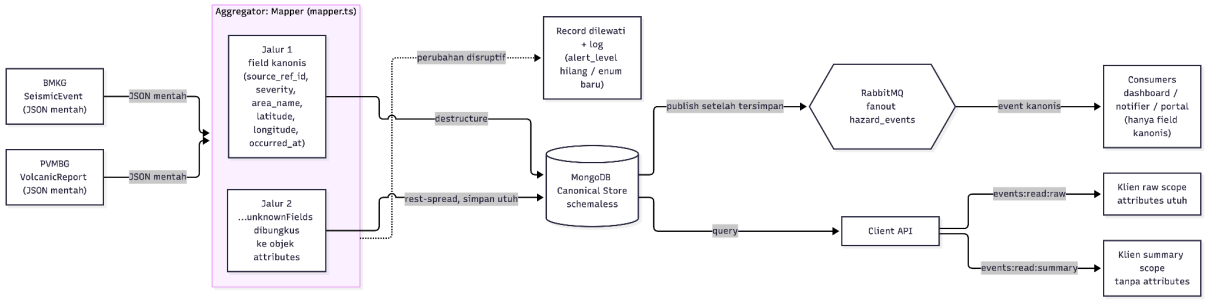
```
docker compose up -d --build
```

3. *Problem Set*

3.1. *Problem* 1: Arsitektur & Interoperabilitas Data

3.1.1. Rancangan Solusi

Sistem Gateway BNPB di sini dirancang dengan *canonical contract*. Dengan ini, sistem hanya mewajibkan beberapa *field* data utama (seperti `hazard_id`, `severity`, `latitude`, dll). *Field* lain (yang ditambahkan secara sepihak) akan dianggap sebagai *field* yang tidak diketahui yang disimpan dalam objek attributes.



Gambar 3.1.1.1 Diagram Rancangan Solusi *Problem* 1

Sistem mendaftarkan 10 *field* utama yang wajib ada pada `HazardEvent`. Sebagai penangkap *field* tambahan dari pihak sumber, disediakan satu *field* attributes dengan tipe `Record<string, any>` (`mapper.ts:3-15`). Kontrak ini direplikasi di sisi Client API (`client-api/src/types.ts:12`).

```
export interface HazardEvent {
   hazard_id: string;
   source: 'BMKG' | 'PVMBG';
   source_ref_id: string;
   hazard_type: 'SEISMIC' | 'VOLCANIC';
   severity: 'NORMAL' | 'WASPADA' | 'SIAGA' | 'AWAS';
   area_name: string;
   latitude: number;
   longitude: number;
   occurred_at: string;
   ingested_at: string;
   attributes: Record<string, unknown>;
}
```

Pada fungsi `mapVolcanicToHazard` untuk PVMBG dan `mapSeismicToHazard` untuk BMKG, sistem hanya menarik *field* yang sudah disepakati. Sisa data yang tidak dikenali ditangkap menggunakan operator *rest* (`...unknownFields`) di dalam objek `attributes`. (`mapper.ts:45-80,94-127`).

```
// PVMBG
export function mapSeismicToHazard(raw: any, tsunamiWarning?: any):
HazardEvent {
   ...
   return {
       ...
       attributes: {
           ...unknownFields,
           ...
       },
   };
}
// BMKG
export function mapVolcanicToHazard(raw: any): HazardEvent {
   ...
   return {
       ...
       attributes: {
           ...unknownFields,
           ...
       },
   };
}
```

Pada implementasi ini, dipilih MongoDB yang bersifat *schemaless* sebagai *database* dengan beberapa pendekatan sebagai berikut.

- Koleksi `hazards` dibentuk tanpa paksaan skema di level *database* (`mongo.ts:40-43`).

```
export function getHazards(): Collection<HazardEvent> {
   if (!db) throw new Error('MongoDB not connected');
   return db.collection<HazardEvent>('hazards');
}
```

- Indeks hanya dipasang pada *field* utama (tidak pada *attributes*) (`mongo.ts:23-25`).

```
await getHazards().createIndex({ source: 1, source_ref_id: 1 }, { unique:
true });
await getHazards().createIndex({ occurred_at: -1 });
await getHazards().createIndex({ hazard_id: 1 }, { unique: true });
```

- Penyimpanan menggunakan operasi `$set` dengan metode `upsert`. Artinya, dokumen lama dan dokumen baru bisa tersimpan tanpa membutuhkan proses migrasi data (`hazardRepository.ts:26-30`).

```
const result = await hazards.updateOne(
    { source: h.source, source_ref_id: h.source_ref_id },
    { $set: rest, $setOnInsert: { hazard_id, ingested_at } },
    { upsert: true }
);
```

Selain itu, pada proses *polling*, pemrosesan dilakukan satu per satu. Jika ada satu data yang rusak, data tersebut akan dilewati dengan log yang dicatat oleh sistem. Sistem akan lanjut memproses data berikutnya setelah log data rusak tercatat. (`index.ts:238-247`)

```
        const newHazards: HazardEvent[] = [];
        for (const raw of events) {
            try {
                const relatedWarning = pickStrongestWarning(warnings,
raw.event_id);
                newHazards.push(mapSeismicToHazard(raw, relatedWarning));
            } catch (error) {
                log.error({ event_id: raw?.event_id, error:
errorMessage(error) }, 'Skipping malformed seismic record');
            }
        }
```

3.1.2. Alasan Keputusan

Keputusan untuk menggunakan objek attributes sebagai tempat penyimpanan *field* selain wajib (*additive-only*) sesuai dengan masalah yang ada pada *problem* 1. BMKG dan PVMBG dapat mengubah skema secara sepihak tanpa adanya koordinasi dengan BNPB. Dengan desain seperti ini, sistem tidak akan berhenti ketika menerima data dengan penambahan *field* baru. Selain itu, pendekatan ini memungkinkan data yang strukturnya diganti langsung masuk ke *database* tanpa memerlukan *deploy* ulang atau *restart container*. Desain ini sederhana dan aman karena tidak diperlukan pemeliharaan Schema Registry, sistem dapat memanfaatkan *rest-spread* bawaan.

3.1.3. Batasan

Solusi ini memang mengatasi masalah penambahan *field* pada skema oleh lembaga (PVMBG dan BMKG) tanpa koordinasi ke BNPB. Namun, solusi ini masih memiliki beberapa batasan sebagai berikut.

Tabel 3.1.3.1 Batasan Solusi Problem 1

Batasan	Penjelasan	Referensi
Sistem belum mampu menangani perubahan yang bersifat substraktif.	Hanya perubahan yang bersifat aditif (penambahan) yang dapat di-*handle*. Penghapusan atau penggantian *field* yang bersifat wajib membuat *record* tidak valid dan dilewati.	`mapper.ts:82-92, 117` `index.ts:290-292`
Objek `attributes` tidak bertipe, disimpan apa adanya.	Objek `attributes` dideklarasikan sebagai tipe `any` pada sistem dan `unknown` pada *client*. Sistem akan menerima data secara mentah tanpa melakukan validasi tipe (sifatnya hanya sebagai penyimpan).	`mapper.ts:14` `types.ts:12`
Belum ada mekanisme untuk memantau tambahan field baru.	*Field* informasi baru dari instansi sumber masuk ke *database* tanpa dilengkapi *logging* dan/atau mekanisme lain yang dapat mengidentifikasi kemunculan *field* baru.	`index.ts:285-293`

3.1.4. Bukti Penyelesaian

Sistem terbukti mampu menangani perubahan skema aditif secara dinamis dari instansi sumber tanpa memerlukan migrasi data ataupun *restart* pada layanan. Dapat ditelusuri pada kode sumber, tidak ada bagian yang memuat langkah migrasi, DDL (Data Definition Language), dan penggunaan ORM relasional. Dapat dilihat pada tangkapan layar di bawah,

hasil pencarian kode (`grep`) membuktikan tidak adanya skrip migrasi, dan konfigurasi *collection* MongoDB menunjukkan `options: {}`, yang artinya tidak ada validator skema.

```
(base) mac@Kaindras-MacBook-Pro IF4031_M1_RMB % mq 'printjson(db.getCollectionInfos({name:"hazards"}).map(c => ({name:c.name, options:c.options})))'
[
  {
    name: 'hazards',
    options: {}
  }
]
```

Gambar 3.1.4.1 Pencarian terhadap *Collection* `hazards`

Sebelum PVMBG menerapkan perubahan bentuk skema, seluruh *record* PVMBG di dalam basis data (lebih dari 20 data historis) memiliki satu bentuk atribut seragam tanpa keberadaan *field* `confidence_level`.

```
(base) mac@Kaindras-MacBook-Pro IF4031_M1_RMB % mq 'printjson(db.hazards.aggregate([{ $match:{source:"PVMBG"} }, { $group:{_id:{$map:{input:{$objectToArray:"$attributes"},as:"a",in:"$$a.k"}}, jumlah:{$sum:1}} }]).toArray())'
[
  {
    _id: [
      'eruption_count_24h',
      'ash_column_height_m'
    ],
    jumlah: 403
  }
]
(base) mac@Kaindras-MacBook-Pro IF4031_M1_RMB % mq 'printjson({tanpa: db.hazards.countDocuments({source:"PVMBG","attributes.confidence_level":{$exists:false}}), dengan: db.hazards.countDocuments({source:"PVMBG","attributes.confidence_level":{$exists:true}})})'
{
  tanpa: 404,
  dengan: 0
}
```

Gambar 3.1.4.2 Tidak Ada *Record* dengan Atribut `confidence_level`

Setelah *endpoint* `mock-pvmbg` dipicu untuk menambahkan informasi `confidence_level` pada *record* baru, aggregator secara otomatis menyimpan format baru tersebut tanpa kendala. Di dalam *collection* yang sama, kini terdapat dua jenis *record* (dengan dan tanpa atribut baru).

```
(base) mac@Kaindras-MacBook-Pro IF4031_M1_RMB % mq 'printjson(db.hazards.aggregate([{ $match:{source:"PVMBG"} }, { $group:{_id:{$map:{input:{$objectToArray:"$attributes"},as:"a",in:"$$a.k"}}, jumlah:{$sum:1}, contoh:{$first:"$attributes"}} }]).toArray())'
[
  {
    _id: [
      'eruption_count_24h',
      'ash_column_height_m',
      'confidence_level'
    ],
    jumlah: 10,
    contoh: {
      eruption_count_24h: 5,
      ash_column_height_m: 4380.9,
      confidence_level: 0.66
    }
  },
  {
    _id: [
      'eruption_count_24h',
      'ash_column_height_m'
    ],
    jumlah: 409,
    contoh: {
      eruption_count_24h: 8,
      ash_column_height_m: 1876
    }
  }
]
```

Gambar 3.1.4.3 Dua Jenis *Record* yang Disimpan setelah Penambahan Atribut

Hitungan dokumen membuktikan bahwa terdapat data baru dengan jenis atribut baru yang berhasil dicatat.



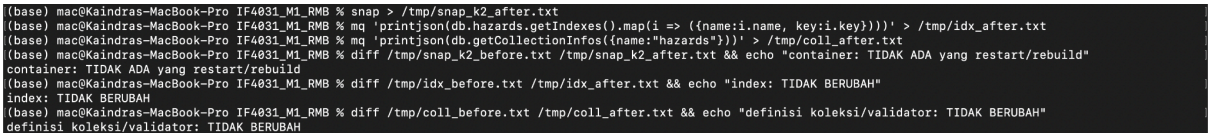
Gambar 3.1.4.4 Terdapat 14 *Record* dengan Atribut `confidence_level`

Kedua jenis *record* tersebut (skema lama dan skema baru) terbukti dapat diakses dan dibaca secara normal melalui Client API.



Gambar 3.1.4.5 Kedua Jenis Skema Dapat Terbaca

Pengecekan *snapshot container*, indeks basis data, dan definisi *collection* dilakukan sebelum dan sesudah perubahan skema terjadi. Output dari perintah diff membuktikan bahwa tidak ada *container* yang di-*restart* atau di-*rebuild*, dan tidak ada indeks atau definisi koleksi yang diubah secara manual.



Gambar 3.1.4.6 Tidak Ada *Restart*, *Rebuild*, maupun Perubahan pada Definisi *Collection*

Dengan ini, dapat disimpulkan bahwa arsitektur Gateway BNPB terbukti mampu beradaptasi terhadap penambahan skema data baru dari instansi sumber secara otomatis, tanpa memerlukan eksekusi migrasi basis data, modifikasi kode, maupun penghentian layanan operasional.

3.1.5. Alternatif Solusi

Terdapat beberapa alternatif pendekatan desain beserta alasan desain tersebut tidak digunakan sebagai berikut.

Tabel 3.1.5.1 Alternatif Desain

Alternatif Desain	Alasan Tidak Digunakan
Skema *Registry* Terpusat	Membutuhkan pendaftaran skema baru setiap kali ada perubahan. Pada kasus ini, BMKG dan PVMBG mengubah data secara sepihak tanpa koordinasi. Desain seperti ini akan membuat bentuk data tidak valid tanpa adanya pendaftaran skema.
Membuang *Field* Tak Dikenal	Pendekatan ini aman untuk memastikan sistem tetap berjalan dan menerima data. Namun BNPB akan kehilangan informasi apabila *field* baru bersifat penting.

Desain tempat penyimpanan objek `attributes` dianggap menjadi solusi yang paling optimal. Keputusan ini diambil karena solusi tersebut merupakan pendekatan yang mendukung untuk kondisi di mana PVMBG dan BMKG dapat melakukan penambahan atribut tanpa koordinasi dengan pihak BNPB.

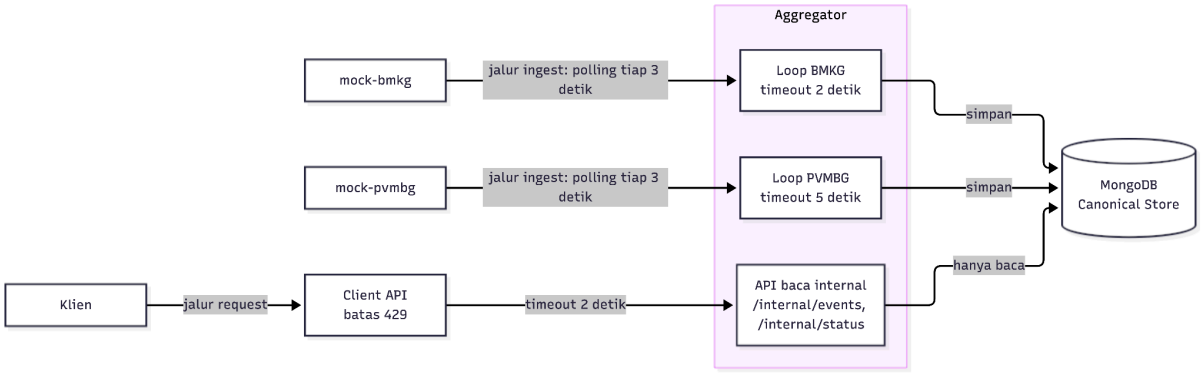
3.2. *Problem* 2: *Concurrency*, *Availability*, & *Graceful Degradation*

3.2.1. Rancangan Solusi

Ide desain utama yang digunakan di sini adalah *store-first*, di mana sistem terbagi menjadi dua jalur yang tidak saling menunggu: Aggregator akan melakukan mengambil data dari *mock* BMKG dan PVMBG di *background* dan menyimpannya pada Canonical Store, lalu Client API akan membaca dari *store* tersebut melalui API internal yang disediakan Aggregator. Dengan desain ini, kecepatan serta matinya *mock* PVMBG tidak akan mengganggu jalur *request*. Selain itu, terdapat detail implementasi lainnya pada **Tabel 3.2.1.1**.

Tabel 3.2.1.1 Detail Mekanisme dan Masalah yang Dicegah

Mekanisme	Masalah yang Dicegah
Loop *polling* yang dipisah antara *mock* BMKG dan PVMBG	*Head-of-line blocking*
Timeout pada seluruh komunikasi eksternal	*Stale request* (menggantung dan menghabiskan *resource*)
Penanda kegagalan per sumber (`last_failure_at` pada `source_status`)	Degradasi yang tidak terkontrol (menargetkan *faithful degradation*)
Penggunaan `Promise.allSettled` dan penanda `stale` / `stale_since`	Degradasi yang tidak terkontrol (menargetkan *faithful degradation*)
Batas jumlah *request* dalam satu waktu (HTTP 429 dan `Retry-After`)	Mencegah kehabisan *resource* (mengurangi *server-side failure*)



Gambar 3.2.1.1 Diagram Rancangan Solusi Problem 2

Untuk mencegah *head-of-line blocking*, dibuat *loop* terpisah per sumber, di mana masing-masing sumber memiliki kursor dan *timer*-nya sendiri. Dengan ini, proses *polling* PVMBG yang lebih lambat tidak akan menahan *polling* BMKG.

```
// aggregator/src/index.ts:34-37, 383
let lastFetchBMKG = new Date(0).toISOString();
let lastFetchPVMBG = new Date(0).toISOString();
const polling = { BMKG: false, PVMBG: false };

const pollers = [setInterval(pollBmkg, POLLING_INTERVAL),
setInterval(pollPvmbg, POLLING_INTERVAL)];

// aggregator/src/index.ts:263-271, 301-303
async function pollPvmbg(): Promise<void> {
    if (polling.PVMBG) { log.warn('Previous PVMBG poll still running,
skipping this tick'); return; }
    polling.PVMBG = true;
    try {
        ...
    } finally {
        polling.PVMBG = false;
    }
}
```

Untuk mencegah *request* yang basi, setiap komunikasi eksternal (API *call*) diterapkan *timeout*. Selain itu, *poll* yang gagal akan langsung dicatat. Ketika Client API yang memanggil data dan status secara paralel, lalu terjadi kegagalan pada satu panggilan, maka hasil lainnya akan tetap disimpan (tidak terjadi kegagalan).

```
export async function markSourceSuccess(source: 'BMKG' | 'PVMBG':
Promise<void> {
    await getSourceStatus().updateOne(
        { _id: source },
        { $set: { last_success_at: new Date().toISOString() }, $unset: {
last_failure_at: '' } },
        { upsert: true }
    );
}

export async function markSourceFailure(source: 'BMKG' | 'PVMBG':
Promise<void> {
    await getSourceStatus().updateOne(
        { _id: source },
        { $set: { last_failure_at: new Date().toISOString() } },
        { upsert: true }
    );
}
```

Terakhir, terdapat batas jumlah *request* bersamaan yang diimplementasikan sebagai sebuah *middleware*.

```
if (inFlight >= max) {
    res.setHeader('Retry-After', '1');
    res.status(429).json({ error: 'Too many requests, please try again
shortly' });
    return;
}
inFlight++;
let released = false;
const release = () => { if (released) return; released = true;
inFlight--; };
res.on('finish', release);
res.on('close', release);
```

3.2.2. Alasan Keputusan

Berikut merupakan masing-masing desain dan alasan keputusannya.

Tabel 3.2.2.1 Desain dan Alasan Keputusan Problem 2

Desain	Alasan	Alternatif (Ditolak)
Store-first	Klien tidak perlu menunggu BMKG dan PVMBG sehingga *mock* yang lambat atau mati tidak memperlambat klien. Data lama masih ada saat *mock* mati.	Memanggil keduanya saat ada *request*: klien harus menunggu paling lambat 3 detik dan tidak ada data sama sekali ketika PVMBG mati.
Loop *polling* terpisah per *mock*	PVMBG yang membutuhkan waktu 3 detik tidak menahan BMKG.	Satu putaran untuk keduanya: BMKG berpotensi ikut terhambat.
HTTP 200 dengan stale dan stale_since saat *mock* gagal	Bagian dari *faithful degradation* dan spesifikasi.	Menjawab 503 untuk seluruh *request* vulkanik.
Kegagalan dicatat pada *poll* pertama yang gagal	PVMBG langsung terlihat *stale* sekitar 3 detik dan tidak perlu menunggu 10 detik	Hanya melihat umur sukses terakhir: baru muncul setelah 10 detik.
Timeout	BMKG biasanya menjawab dalam 50-150 ms, dan PVMBG hingga 3 detik, dengan Aggregator hanya membaca MongoDB.	Tanpa batas waktu: *request* menggantung dan boros memori.

Desain	Alasan	Alternatif (Ditolak)
Batas 100 *request* bersamaan	Lebih baik menolak dengan cepat dan memberi tahu kapan mencoba lagi daripada membuat keseluruhan sistem kewalahan.	Antrean tanpa batas: masalah pindah ke memori.

3.2.3. Batasan

Berikut merupakan batasan desain beserta deskripsi dan alasannya:

Tabel 3.2.3.1 Batasan Implementasi Sistem Problem 2

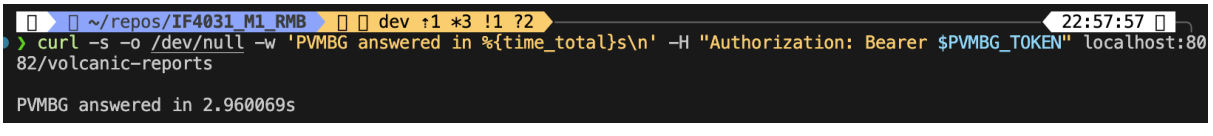
Batasan	Deskripsi dan Alasan
Freshness PVMBG dibatasi interval *polling*	Data yang baru terlihat setelah *polling interval* 3 detik yang mana ini merupakan *tradeoff* dari *store-first* agar klien tidak pernah menunggu sumber dan *freshness* terlihat dari last_success_at
Aggregator merupakan *single point of failure* untuk baca	Client API sengaja tidak memiliki *copy* karena secara langsung melanggar spesifikasi sistem sehingga /events mengembalikan HTTP 503 bila Aggregator mati

3.2.4. Bukti Penyelesaian

Setelah *delay* PVMBG diatur menjadi 3 detik (`PVMBG_DELAY_MIN_MS=3000` dan `PVMBG_DELAY_MAX_MS=3000`), pengukuran langsung ke mock menunjukkan jawaban sekitar 2,8–3,0 detik per panggilan seperti yang terlihat pada **Gambar 3.2.4.1** dan **Gambar 3.2.4.2**.



Gambar 3.2.4.1 Pengecekan *Delay* PVMBG Pertama



Gambar 3.2.4.2 Pengecekan *Delay* PVMBG Kedua

Hasil seluruh *load-test* dapat dilihat pada `/load-test/results`. Berikut merupakan hasil kunci dari kriteria p95 BMKG-only di bawah < 300 ms dengan *delay* PVMBG 3 detik (`/Load-test/results/final-delay3s.txt`).

```
http_req_duration{scenario:bmkg_only,endpoint:events}
✓ 'p(95)<300' p(95)=82.93ms

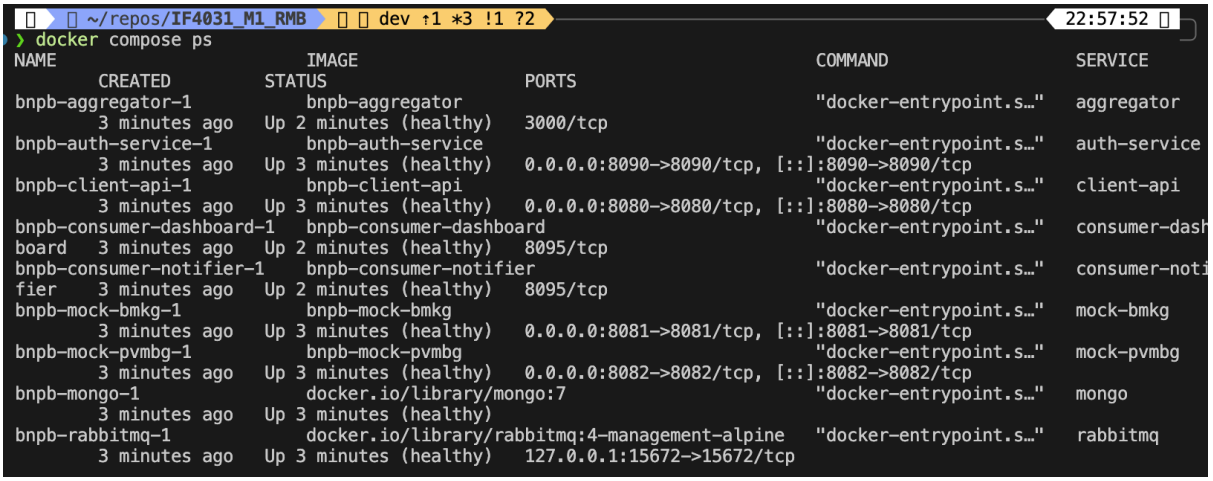
http_req_failed
✓ 'rate<0.01' rate=0.00%
```

Untuk *load-testing* dengan lebih dari 50 koneksi paralel selama minimal 60 detik, pengujian dilakukan sebanyak 3 kali dengan *script* yang sama (`/load-test/events.js`) dengan konfigurasi lengkap pada bagian selanjutnya.

Tabel 3.2.4.1 Hasil Uji Beban 50 Koneksi selama 60 Detik

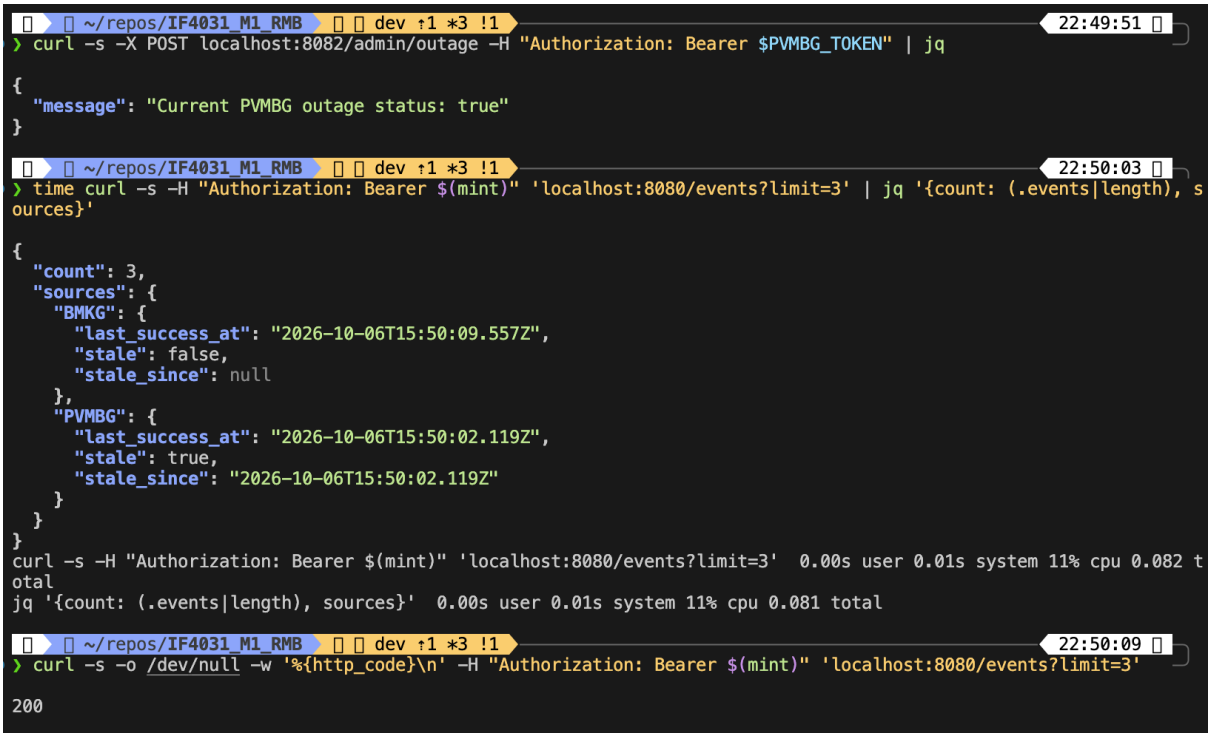
Metrik	Delay 3 Detik	Outage PVMBG	Beban Berlebih
Koneksi (VU)	50	50	200
`MAX_IN_FLIGHT`	100	100	5
Total *request*	57.853	58.620	297.527
Throughput	963,7 req/s	976,5 req/s	4.955,5 req/s
p50 / p95 / p99 (semua *request*)	46,6 / 83,2 / 138,6 ms	46,3 / 80,4 / 134,0 ms	33,9 / 67,3 / 143,0 ms
Latensi maksimum	570 ms	1,01 s	4,44 s
p95 BMKG-only	82,9 ms	80,5 ms	65,9 ms
Error rate	0% (dari 57.853)	0% (dari 58.620)	0% (dari 297.527)
Jumlah HTTP 429	0	0	294 300
Respons vulkanik *stale*	0% (dari 28.882)	100% (dari 29.258)	0% (dari 1.386)
Container crash	Tidak	Tidak	Tidak

Setelah *load-testing*, seluruh *container* tetap *healthy* dan tidak ada yang mengalami *restart*:



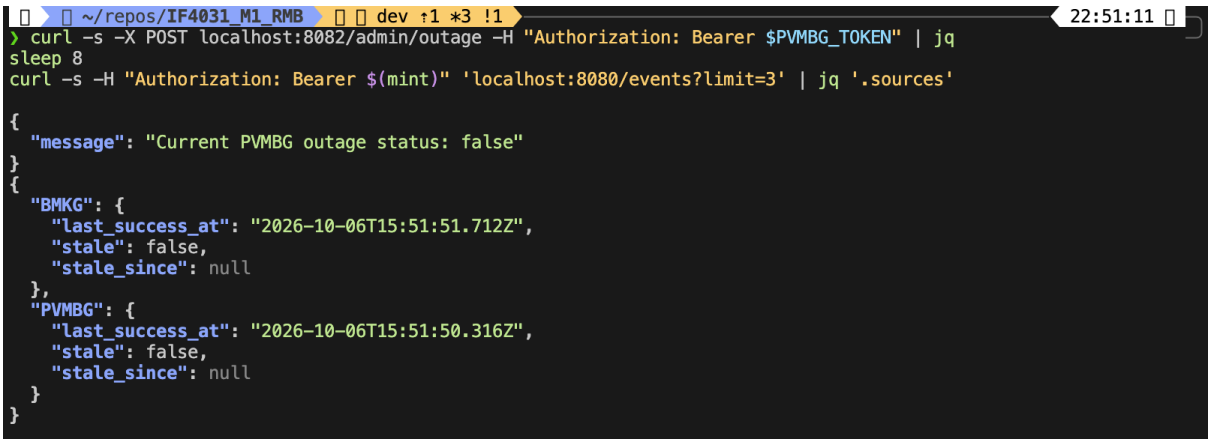
Gambar 3.2.4.3 *Container* setelah *Load Testing*

Pengujian *outage* dilakukan dengan memanggil `POST /admin/outage`, lalu `/events` juga dipanggil. Server pun menjawab 200 dalam 0,08 detik dengan PVMBG yang ditandai `stale: true` dengan `stale_since` yang valid, sedangkan BMKG tetap menyala dan tetap tersaji dari *store*.



Gambar 3.2.4.4 *Outage Testing*

Lalu, *outage* dimatikan tanpa melakukan *restart*, dan delapan detik kemudian PVMBG kembali menyala (`state: false, stale_since: null`).



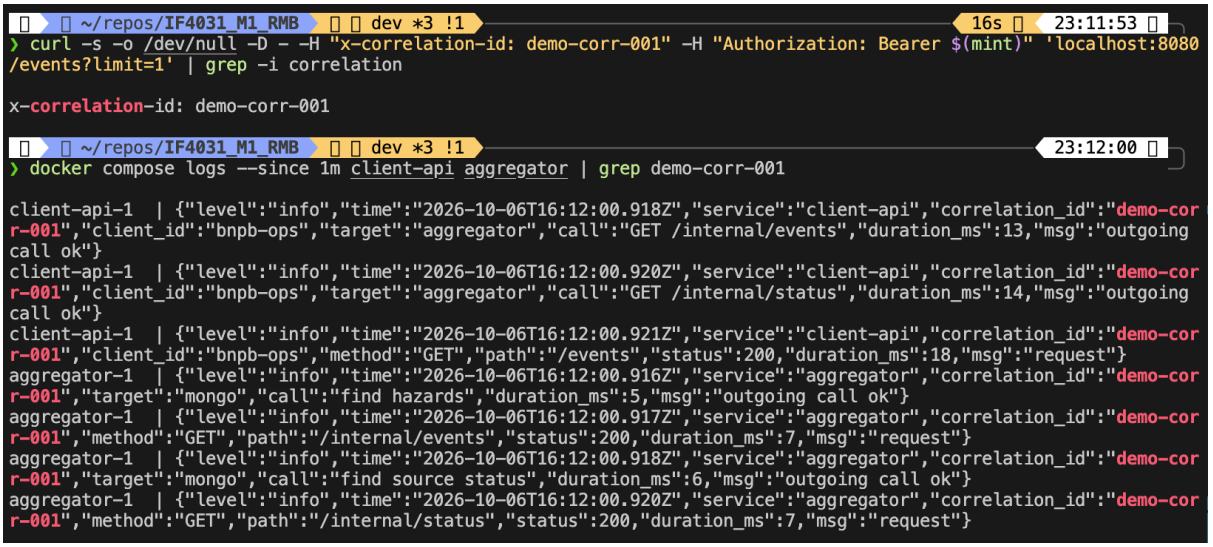
Gambar 3.2.4.5 *Post-Outage Testing*

Saat Aggregator dihentikan, Client API menjawab 503 dalam ~0,067 detik dengan *body* yang jelas dan `correlation_id`. `/health` Client API tetap 200 and tidak ikut mati.



Gambar 3.2.4.6 Aggregator Tidak Tersedia

Sebagai salah satu sarana observabilitas, *request* selalu memuat `x-correlation-id` pada *header* seperti yang terlihat pada **Gambar 3.2.4.7**, yang pada contoh ini muncul pada kedua log Client API dan Aggregator. Log juga mencatat *latency* tiap panggilan eksternal (`duration_ms`), yaitu Client API ke Aggregator dan Aggregator ke Canonical Store (MongoDB).



Gambar 3.2.4.7 *Correlation ID Visibility*

3.2.5. Pembahasan: Konkurensi, Pencegahan Kegagalan, dan *Load Testing*

Seperti yang sudah dibahas pada bagian Rancangan Solusi, konsekuensi dan pencegahan kegagalan diterapkan dengan pemilihan model model *store-first* dengan *polling* terpisah per sumber yang berjalan di atas *event loop* Node.js dan dibatasi oleh jumlah *request* bersamaan.

Untuk *loading testing* dan *tooling*, berikut merupakan penjabarannya:

Tabel 3.2.5.1 Konfigurasi *Load Testing* dan *Tooling*

Item	Nilai
Tool	k6 v2.3.0
Skenario	`bmkg_only` (`GET /events?hazard_type=SEISMIC&limit=20`) dan `volcanic` (`GET /events?hazard_type=VOLCANIC&limit=20`)
Koneksi dan Durasi	2 x 25 *virtual user* (VU), 60 detik
Autentikasi	Satu token per VU yang diperbarui melalui `/refresh` sebelum TTL 60 detik
Ambang Batas	P95 BMKG-only di bawah 300 ms, `http_req_failed` di bawah 1%
Penolakan Terkontrol	HTTP 429 dianggap respons yang diharapkan
Metrik Tambahan	`state_volcanic`: proporsi respons vulkanik dengan `PVMBG.stale = true` terhadap total respons vulkanik keseluruhan

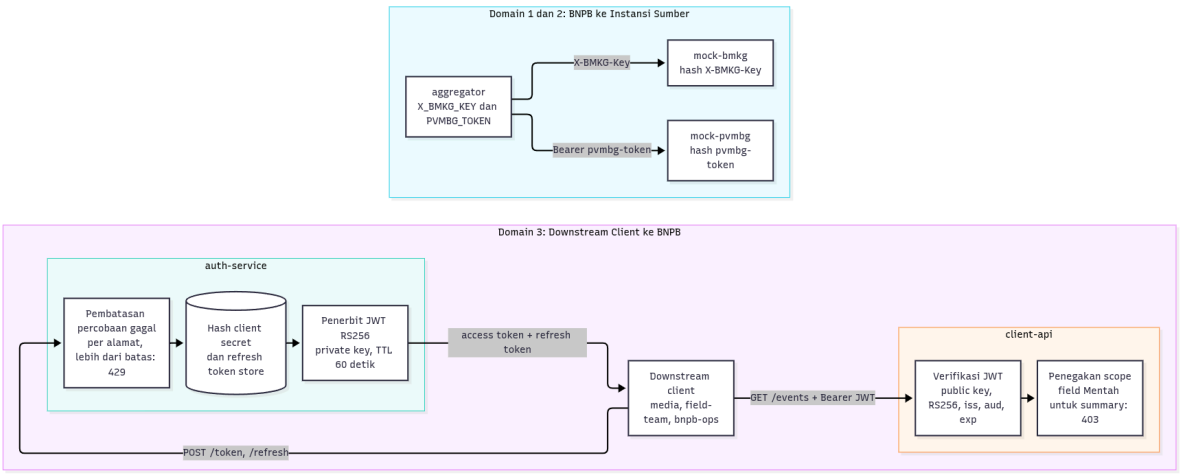
Item	Nilai
Environment	Seluruh *stack* lewat docker compose, PVMBG pada 3000 ms

Dua skenario sengaja dijalankan bersamaan agar *request* yang tidak membutuhkan PVMBG diukur saat *request* yang membutuhkannya sedang berlangsung. Hal ini dilakukan untuk memastikan p95 rendah pada `bmkg_only` bukan hasil jalur yang sepi. Walaupun HTTP 429 dipisah dari kegagalan, jumlahnya tetap dirancang agar dapat dilaporkan dan dipisahkan dari kegagalan yang sesungguhnya (5xx atau koneksi gagal). Metrik `state_volcanic` menunjukkan apakah PVMBG yang lambat atau mati memengaruhi *data freshness*.

3.3. *Problem* 3: *Service-to-Service Authentication & Trust*

3.3.1. Rancangan Solusi

Sistem dibagi menjadi tiga *domain of trust* dengan kredensial yang independen, yaitu BNPB ke BMKG, BNPB ke PVMBG, dan *downstream client* ke BNPB. **Gambar 3.3.1.1** memperlihatkan ketiga domain tersebut beserta komponen yang menerbitkan dan memverifikasi kredensial pada masing-masing domain.



Gambar 3.3.1.1 Diagram Rancangan Solusi *Problem* 3

Tabel 3.3.1.1 memuat kredensial, pemegang, dan *verifier* pada setiap domain.

Tabel 3.3.1.1 Pembagian *Domain of Trust*

Domain	Kredensial	Pemegang	*Verifier*
BNPB ke BMKG	*API key* pada header X-BMKG-Key berformat `bmkg_<64 hex>`	Aggregator	mock-bmkg
BNPB ke PVMBG	*Bearer token* berformat `pvmbg.v1.<43 karakter base64url>`	Aggregator	mock-pvmbg
Downstream client ke BNPB	*Client ID* dan *client secret* yang ditukar menjadi JWT RS256	Setiap *client*	Auth Service (secret) dan Client API (JWT)

Kredensial kedua *mock service* memiliki format dan nilai yang berbeda. *Mock service* hanya menyimpan *hash* SHA-256 dan membandingkannya menggunakan *constant-time comparison*. Kredensial BMKG yang dikirimkan ke PVMBG tidak memenuhi format *header* maupun nilai *hash* yang diharapkan, sehingga ditolak dengan HTTP 401, demikian pula berlaku sebaliknya.

Auth Service mendaftarkan tiga *client* dengan *secret* yang berbeda. *Client* `media` memperoleh *scope* `events:read:summary`, sedangkan `field-team` dan `bnpb-ops` memperoleh *scope* `events:read:raw`. *Auth Service* hanya menyimpan *hash* dari *secret* setiap *client*, menolak dinyalakan apabila dua *client* memiliki *secret* yang sama, dan tetap membandingkan *client ID* yang tidak terdaftar dengan sebuah *dummy hash* agar keberadaan suatu identitas tidak dapat ditebak dari waktu respons.

Setelah *client* berhasil diautentikasi, *Auth Service* menerbitkan *access token* berupa JWT RS256 yang memuat *claim issuer, audience, subject, scope,* identitas *token,* serta waktu kedaluwarsa dengan TTL *default* 60 detik.

Client API hanya memegang *public key*. *Token* diverifikasi dengan algoritma yang dikunci pada RS256, serta pemeriksaan *issuer, audience,* dan *claim* wajib. *Token* yang kedaluwarsa ditolak dengan HTTP 401 dan kode *error* `token_expired`.

```
const { payload } = await jwtVerify(header.slice('Bearer '.length),
publicKey, {
   algorithms: ['RS256'],
   issuer: config.jwtIssuer,
   audience: config.jwtAudience,
   requiredClaims: ['sub', 'exp', 'scope'],
});
...
if (error instanceof errors.JWTExpired) {
   reject(res, 401, 'token_expired', 'Access token has expired, please
          refresh');
   return;
}
```

Pembatasan hak akses di-*enforce* di sisi *server*. *Scope* ringkasan hanya menerima *field* berklasifikasi Ringkasan, dan permintaan eksplisit terhadap *field* Mentah (`source_ref_id`, `latitude`, `longitude`, `attributes`) melalui parameter `fields` ditolak dengan kode HTTP 403.

Alur pembaruan *token* menggunakan *refresh token* dengan rotasi. Setiap pemanggilan `POST /refresh` menghasilkan pasangan *token* baru dan menandai *refresh token* lama sebagai

terpakai. Jika *refresh token* yang sudah terpakai digunakan kembali, *Auth Service* menganggap hal tersebut sebagai indikasi pencurian token dan mencabut seluruh *token family* yang bersangkutan.

```
if (entry.used) {
   revokeFamily(entry.family);
    return { ok: false, reason: "reused", clientId: entry.clientId };
}
if (entry.expiresAt <= Date.now()) {
    revokeFamily(entry.family);
    return { ok: false, reason: "expired", clientId: entry.clientId };
}

entry.used = true;
```

Untuk menghambat *brute force*, percobaan yang gagal pada `POST /token` dan `POST /refresh` dihitung per alamat asal. Setelah 10 percobaan gagal dalam jendela 60 detik (keduanya dapat dikonfigurasi), permintaan berikutnya dari alamat tersebut dijawab dengan HTTP 429 beserta *header* `Retry-After`.

Untuk setup awal, seluruh kredensial dibangkitkan oleh `scripts/gen-secrets.sh` ke *file* `.env` yang di-*gitignore*. Repositori hanya menyertakan `.env.example` berisi nilai contoh, dan Docker Compose menolak menyalakan *service* apabila kredensial yang diperlukan belum diisi melalui sintaks `${VAR:?}`.

3.3.2. Alasan Keputusan

Berikut merupakan masing-masing desain dan alasan keputusannya.

Tabel 3.3.2.1 Desain dan Alasan Keputusan *Problem* 3

Desain	Alasan	Alternatif (Ditolak)
JWT RS256	Hanya *Auth Service* yang memegang *private key*, sehingga *verifier* tidak dapat menerbitkan *token* sendiri.	JWT HS256, dimana setiap *verifier* memegang *shared secret* dan sekaligus dapat menerbitkan *token*.
Verifikasi *token* secara lokal di *Client API*	Tidak ada *network call* tambahan per *request,* sejalan dengan Problem 2.	*Opaque token* dengan introspeksi yang dapat dicabut kapanpun, tetapi setiap *request* bergantung pada *Auth Service*.

Desain	Alasan	Alternatif (Ditolak)
Scope di-*enforce* di sisi *server*	Hak akses tidak bergantung pada kepatuhan *client*.	Menyembunyikan *field* Mentah secara *client-side*.
Refresh token dengan rotasi dan *reuse detection*	Pencurian *refresh token* dapat dideteksi, dan *client* tidak perlu *login* ulang secara manual.	*Refresh token* statis berumur panjang.
Pembatasan percobaan per alamat	Menghambat *brute force*. Percobaan yang berhasil tidak dihitung, dan *client* dari alamat lain tidak terdampak.	Penguncian akun setelah beberapa kali gagal, dengan kelemahan berupa *client* yang sah dapat terkunci oleh aksi *attacker*.

Alternatif lain yang dipertimbangkan adalah *mutual* TLS (mTLS), di mana setiap service saling memverifikasi sertifikat yang ditandatangani CA internal. Berbeda dengan kredensial di *header* yang dapat langsung dipakai siapa pun jika bocor, sertifikat tidak berguna tanpa *private key*-nya, dan *private key* tersebut tidak pernah dikirim melalui jaringan. mTLS juga mengenkripsi komunikasi antar-*container* dan mengautentikasi pemanggil *endpoint* internal. Namun, mTLS tidak diterapkan karena kontrak *mock service* menetapkan kredensial berbasis *header* dan kompleksitas pengelolaan sertifikat terlalu besar untuk skala Milestone 1.

Tabel 3.3.2.1 sudah menunjukkan bagaimana kebocoran kredensial, *compromised Client API*, pencurian *refresh token*, dan *brute force* ditangani. Selain itu, **Tabel 3.3.2.2** memetakan ancaman lain yang relevan dengan mitigasinya, sehingga mekanisme yang digunakan disini tidak membuka celah baru.

Tabel 3.3.2.2 Ancaman dan Mitigasinya

Ancaman	Mitigasi
Algorithm confusion (`alg: none` atau HS256 dengan public key)	*Client API* menetapkan daftar algoritma yang diterima di sisi *server* (`algorithms: ['RS256']`), sehingga *header* `alg` pada *token* tidak dipercaya.
Token dari issuer lain atau untuk audience lain	Ditolak melalui pemeriksaan claim `iss` dan `aud`.
Timing attack dan enumerasi *client ID*	Perbandingan berwaktu konstan, dan *client ID* yang tidak terdaftar tetap dibandingkan dengan *hash* tiruan.

3.3.3. Batasan

Berikut merupakan daftar batasan yang masih ada dalam sistem.

Tabel 3.3.3.1 Batasan Implementasi Sistem *Problem* 3

Batasan	Deskripsi & Alasan
Access token tidak dapat dicabut sebelum kedaluwarsa	Tidak ada *denylist*, sehingga *access token* lama tetap berlaku hingga waktu `exp` setelah *refresh*. Yang langsung ditolak adalah *refresh token* lama. Batasan ini dimitigasi dengan TTL pendek 60 detik.
Refresh token disimpan di memori	*Restart Auth Service* mengharuskan seluruh *client* melakukan *login* ulang, dan *Auth Service* belum dapat diskalakan secara horizontal.
Counter percobaan gagal disimpan di memori per alamat	*Counter* hilang saat *Auth Service* di-*restart*, dan *client* di balik NAT yang sama berbagi batas percobaan.
Tidak ada rotasi *signing key*	Belum terdapat rotasi kunci otomatis maupun *endpoint* JWKS.
Komunikasi antar-*container* tidak terenkripsi	Panggilan antar-*service* di dalam *network* Docker masih menggunakan HTTP biasa.

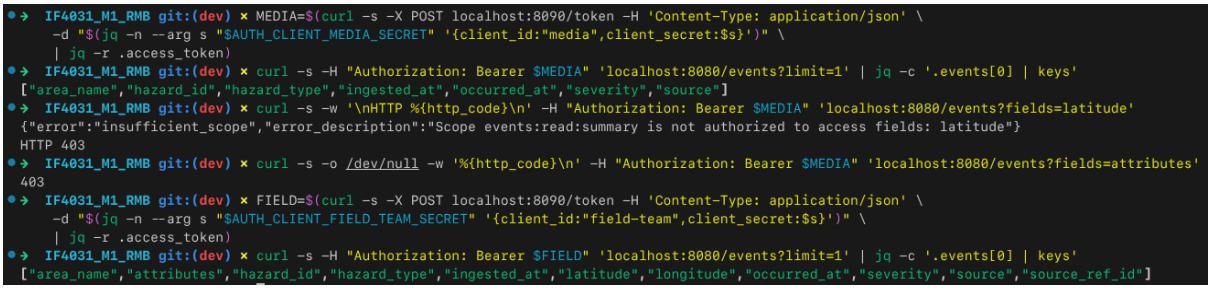
3.3.4. Bukti Penyelesaian

Pengujian isolasi kredensial dilakukan dengan mengirim kredensial milik satu instansi ke instansi lainnya. API key BMKG yang dikirim ke PVMBG dan token PVMBG yang dikirim ke BMKG sama-sama ditolak dengan HTTP 401, begitu pula permintaan tanpa kredensial. Kredensial yang dikirim ke instansi pemiliknya diterima dengan HTTP 200, seperti yang terlihat pada **Gambar 3.3.4.1**.



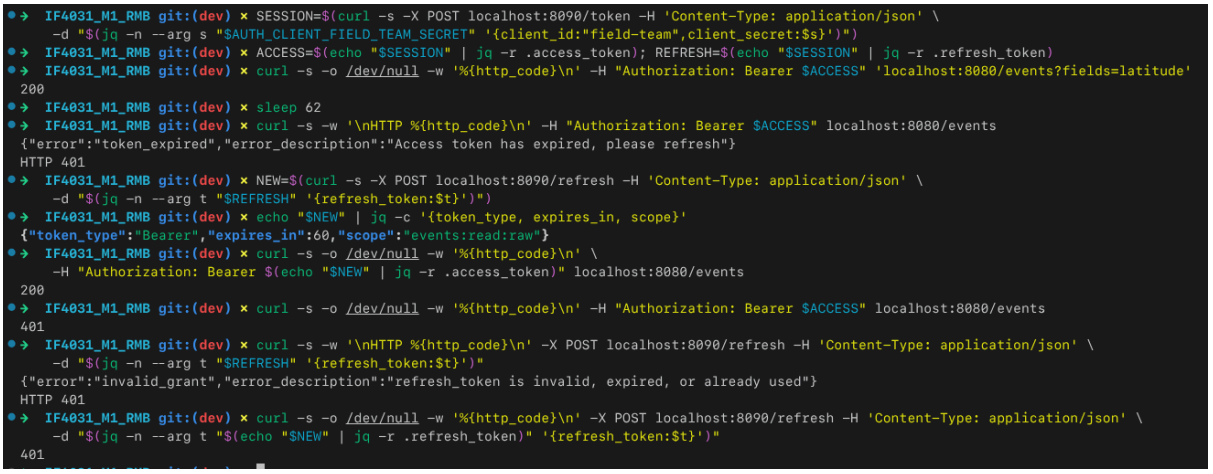
Gambar 3.3.4.1 Bukti Isolasi Kredensial Antarinstansi

Untuk menguji pembatasan *field*, *token* Media digunakan untuk memanggil `/events`. Respons hanya memuat tujuh *field* Ringkasan, dan permintaan eksplisit terhadap *field* Mentah seperti `latitude` dan `attributes` ditolak dengan HTTP 403 beserta *error* `insufficient_scope`. Sebagai pembanding, *token* Tim Lapangan menerima seluruh 11 *field*, termasuk *field* Mentah, seperti yang terlihat pada **Gambar 3.3.4.2**.



Gambar 3.3.4.2 Bukti Pembatasan *Field* untuk *Scope* Media

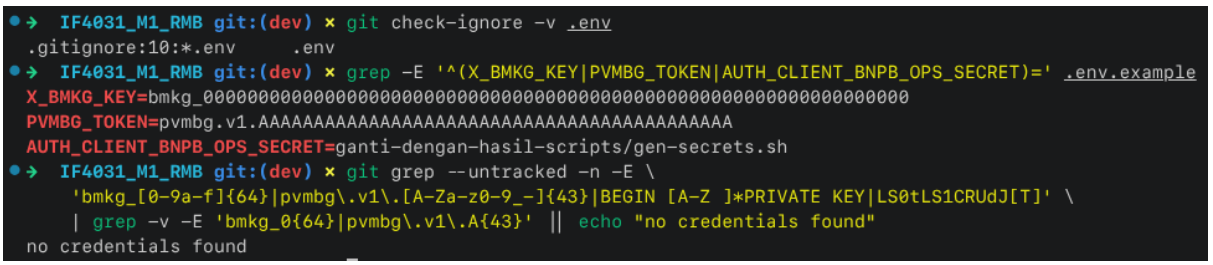
Pengujian kedaluwarsa *token* dilakukan dengan *login* sebagai Tim Lapangan, lalu menunggu selama 62 detik. *Access token* yang semula diterima dengan HTTP 200 kemudian ditolak dengan HTTP 401 dan *error* `token_expired`. Selain itu, *refresh* menghasilkan *token* baru dengan `expires_in` 60 detik yang langsung diterima dengan HTTP 200 tanpa *login* ulang. *Access token* lama tetap ditolak karena sudah kedaluwarsa. *Refresh token* lama juga ditolak dengan *error* `invalid_grant` dan penggunaan ulang tersebut ikut mencabut *refresh token* yang baru, seperti yang terlihat pada **Gambar 3.3.4.3**.



Gambar 3.3.4.3 Bukti Kedaluwarsa dan *Refresh Token*

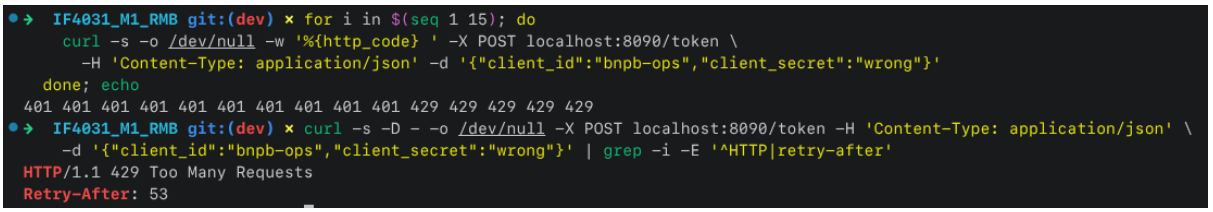
Untuk memastikan tidak-ada kredensial yang ter-*commit*, dilakukan pengecekan terhadap `.gitignore`, `.env.example`, dan isi repositori. File `.env` diabaikan oleh Git, `.env.example` hanya berisi nilai contoh, dan pencarian pola kredensial asli (*API key*

BMKG, *token* PVMBG, dan *private key*) pada repositori tidak menemukan hasil, seperti yang terlihat pada **Gambar 3.3.4.4**.



Gambar 3.3.4.4 Bukti Tidak Ada Kredensial yang Ter-*commit*

Terakhir, dilakukan 15 percobaan *login* dengan *secret* yang salah dari alamat yang sama. Sepuluh percobaan pertama dijawab HTTP 401, sedangkan lima percobaan berikutnya dijawab HTTP 429 dengan *header* `Retry-After`, seperti yang terlihat pada **Gambar 3.3.4.5**.



Gambar 3.3.4.5 Bukti Pembatasan Percobaan Login

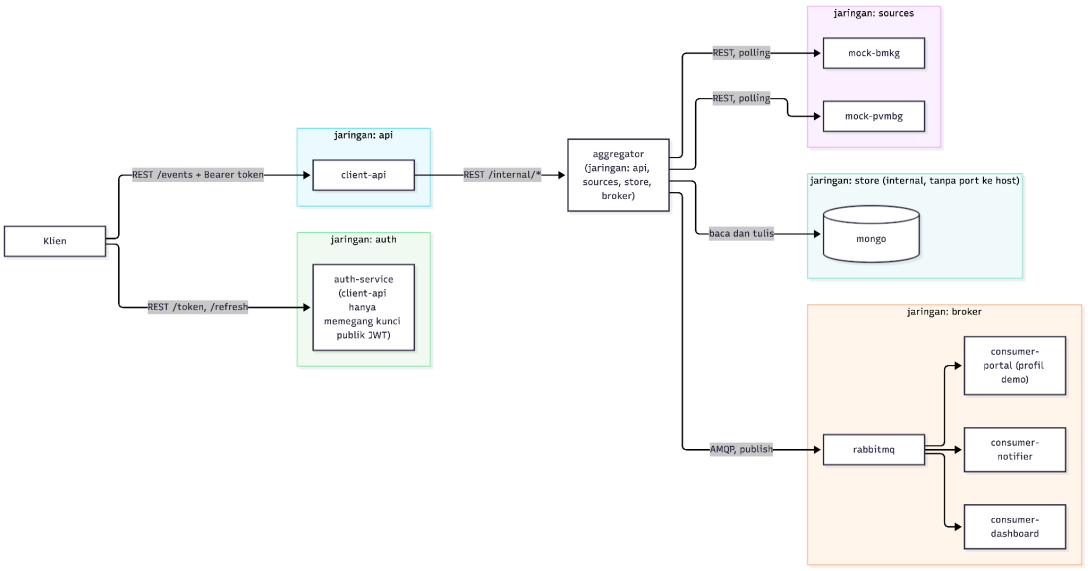
3.4. *Problem* 4: Arsitektur *Microservice* & *Flexible Storage*

3.4.1. Rancangan Solusi

Setiap komponen sistem berjalan pada *container*-nya masing-masing dengan Dockerfile dan konfigurasi JavaScript/TypeScript (seperti `package.json`) yang terpisah. Seluruh komponen berkomunikasi melalui *network* dengan REST dan *message broker*. Pada kasus ini, REST dipilih karena kesederhanaannya dan bentuk dasar kontrak *mock* yang menyerupai JSON, serta performa yang mencukupi *service-level objective* (SLO).

Tabel 3.4.1.1 Pembagian Batas Container dan Jaringan

Komponen	Container	Jaringan
Mock BMKG / PVMBG	`mock-bmkg`, `mock-pvmbg`	`sources`
Aggregator	`aggregator`	`api, sources, store, broker`
Canonical Store	`mongo`	`store (internal)`
Client-Facing API	`client-api`	`api`
Auth Service	`auth-service`	`auth`
Message Broker	`rabbitmq`	`broker`
Event Consumer	`consumer-notifier`, `consumer-dashboard`	`broker`



Gambar 3.4.1.1 Diagram Rancangan Solusi Problem 4

Masing-masing *network* didefinisikan pada `docker-compose.yml`. Pada kasus ini, *network* `store` bersifat internal dan hanya `mongo` serta Aggregator yang tergabung di dalamnya.

```
networks:
  sources:
  api:
  auth:
  broker:
  store:
    internal: true
```

Canonical Store yang berupa MongoDB menggunakan *collection* `hazards`, dengan sekumpulan *fields* tetap (sesuai kontrak) dan tambahan *attributes* untuk *fields* tambahan lainnya seperti `confidence_level`. Dengan menggunakan MongoDB, implementasi ini menjadi mudah tanpa perlu mengubah kode maupun skema.

```
const { report_id, volcano_id, alert_level, reported_at, ...unknownFields
} = raw;
...
attributes: {
    ...unknownFields,
},
```

Pada *resource* MongoDB yang tunggal, seluruh data disimpan dalam format JSONB sehingga perlu dibentuk indeks saat Aggregator terhubung: (`source, source_ref_id`), `occured_at`, dan `hazard_id` yang unik.

```
await getHazards().createIndex({ source: 1, source_ref_id: 1 }, { unique:
true });
await getHazards().createIndex({ occurred_at: -1 });
await getHazards().createIndex({ hazard_id: 1 }, { unique: true });
```

Canonical Store bersifat idempoten dengan *key* (`source, source_ref_id`) yang dipakai untuk *upsert* sehingga *record* yang diambil berulang tetap satu dokumen dan tidak menghasilkan duplikat. *Key* `hazard_id` dan `ingested_at` hanya ditulis saat pertama kali *insert* dilakukan. Terdapat juga *collection* kedua yaitu `source_status` berisi satu dokumen per sumber (`last_success_a, last_failure_at`) sebagai penanda *freshness*.

```
export async function upsertHazard(h: HazardEvent): Promise<boolean> {
    const hazards = getHazards();
    const { hazard_id, ingested_at, ...rest } = h;
    const result = await hazards.updateOne(
        { source: h.source, source_ref_id: h.source_ref_id },
        { $set: rest, $setOnInsert: { hazard_id, ingested_at } },
        { upsert: true }
    );
    return result.upsertedCount > 0;
}
```

Komponen lain tidak membuka koneksi ke Canonical Store dan Client API menggunakan dua *endpoint* internal Aggregator: `GET /internal/events` untuk GET `HazardEvent` dan `GET /internal/status` untuk get *freshness* status dari source_status. Kedua *endpoint* hanya dapat dijangkau melalui *network* `api` (Aggregator hanya membuka `3000/tcp` di dalam *network*).

Sebagai *microservice*, masing-masing *service* dapat dihentikan, di-*build*, dan di-*run* ulang secara sendirian dengan `–no-deps`, tanpa menyentuh container lain.

```
docker compose stop <service-name>
docker compose build <service-name>
docker compose up -d --no-deps <service-name>
```

3.4.2. Alasan Keputusan

Berikut merupakan masing-masing desain dan alasan keputusannya:

Tabel 3.4.2.1 Desain dan Alasan Keputusan Problem 4

Desain	Alasan	Alternatif (Ditolak)
Aggregator memiliki ingest, store, dan API baca	*Polling*, *mapping*, dan *write* ke Canonical Store saling terkait erat. Dengan satu pemilik tinggal, tidak ada penulis lain yang dapat merusak aturan *idempotency.*	Memisahkan Ingestor dan Store API, menambahkan redundansi tanpa manfaat tambahan
Client API tidak boleh mengakses Canonical Store	Menerapkan *separation-of-concern* dan aturan kepemilikan: satu penyimpanan, satu pemilik. Client API hanya perlu mengetahui kontrak sehingga perubahan skema tidak memaksa *deploy* ulang Client API.	Client API membaca langsung dari Canonical Store: lebih cepat tapi melanggar kepemilikan dan mengikat skema

Desain	Alasan	Alternatif (Ditolak)
Auth *service yang* terpisah	Perubahan mekanisme autentikasi tidak boleh memaksa *deploy* ulang layanan lainnya.	Auth sebagai middleware di Client API: setiap perubahan auth memerlukan *deployment* ulang Client API
Consumer terpisah dari Aggregator	*Message broker* memutus ketergantungan: *consumer* dapat mati, ditambah, atau di-*deploy* ulang tanpa menyentuh *producer.*	Aggregator memanggil *consumer* langsung lewat REST: *producer* harus mengenal setiap *consumer*
Network yang dipisahkan berdasarkan tugasnya	Hak jangkau dibatasi oleh topologi. Komponen yang berada di satu jaringan bahkan tidak dapat meresolusi nama layanan lainnya yang tidak berada pada *network* yang sama.	Satu jaringan datar di mana seluruh *service* dapat menjangkau seluruh layanan
Pemisahan *insert* dan *upsert*	*Key* `hazard_id` dan `ingested_at` hanya menerapkan *insert* dan (`source, source_ref_id`) hanya menerapkan *upsert*. Hal ini memungkinan *idempotency* karena identitas hanya disimpan sekali (*insert*) dan *record* yang sama dapat dikirim ulang.	Tidak diterapkan mekanisme *deduplication* dan *idempotency*

3.4.3. Batasan

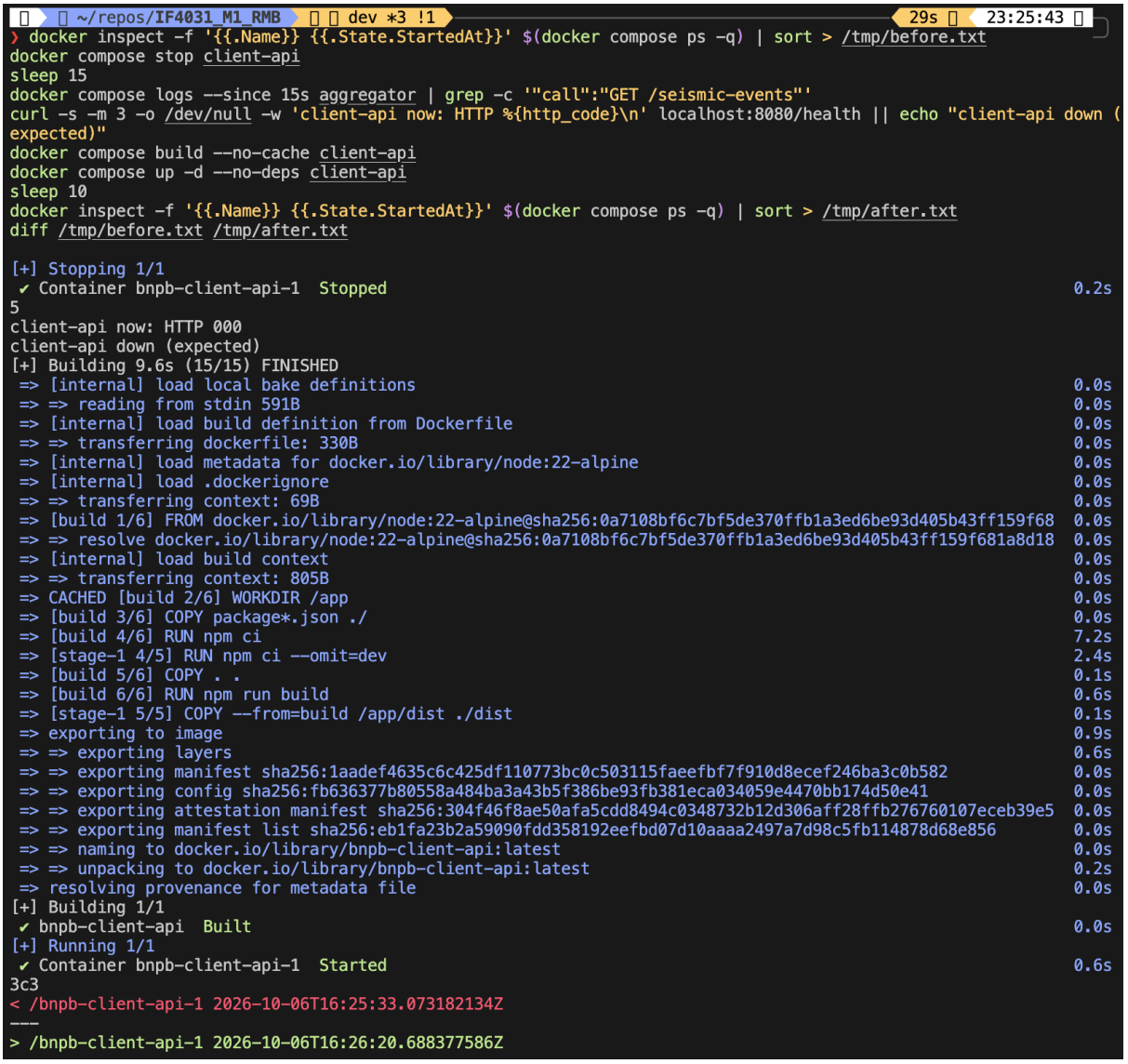
Berikut merupakan daftar batasan yang masih ada dalam sistem.

Tabel 3.4.3.1 Batasan Implementasi Sistem Problem 4

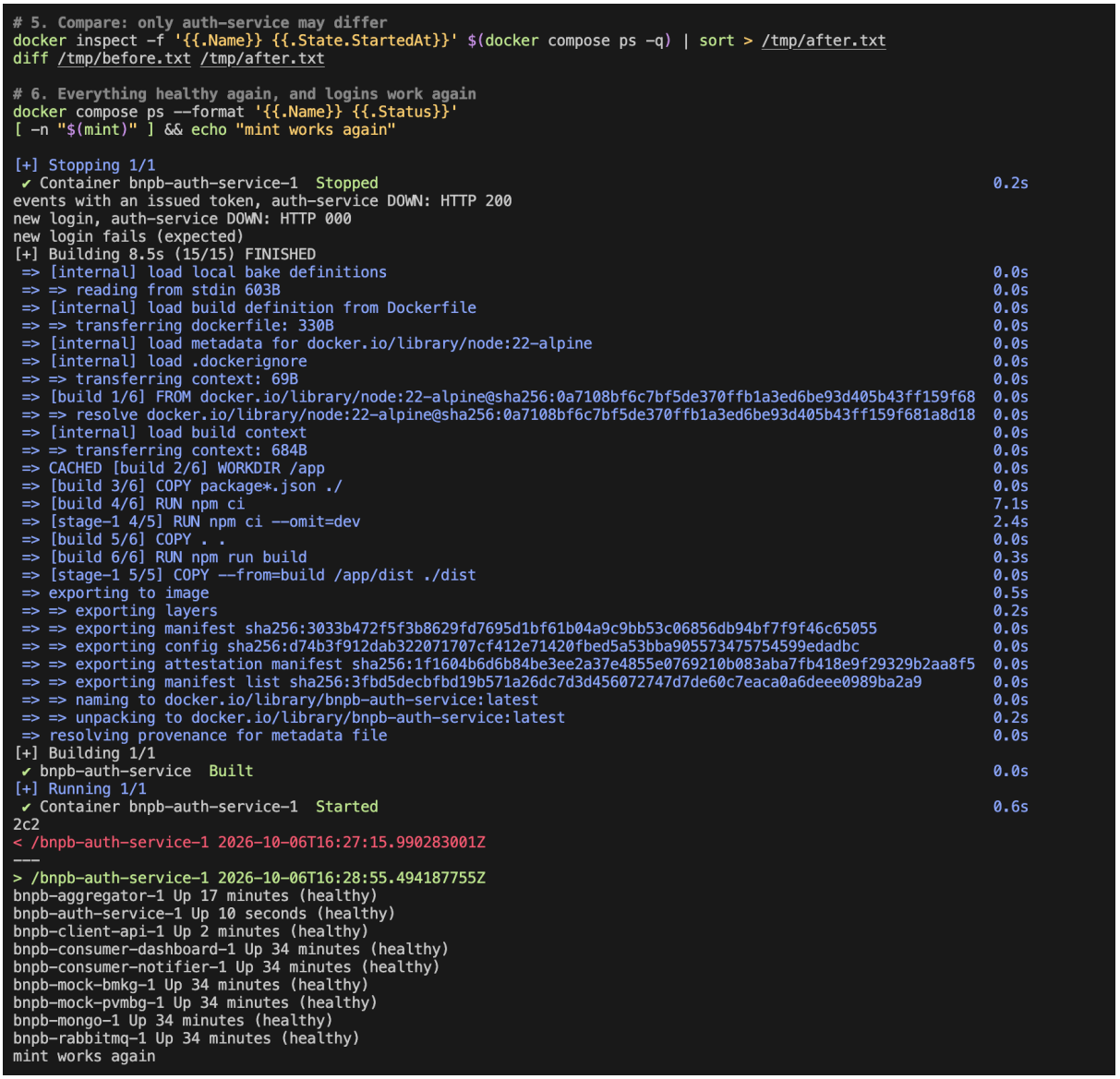
Batasan	Deskripsi	Alasan
MongoDB hanya satu *node*	Tidak ada replikasi atau *backup-store* di luar volume Docker sehingga kegagalan *node* berarti store tidak tersedia	Sistem pada skala saat ini hanya memerlukan satu *node* dan *scope* sistem yang terbatas pada Docker tanpa SLO untuk ketahanan
Tidak ada validasi skema	Perlindungan terhadap *record* diterapkan pada level *mapper*, bukan pada MongoDB	Mengikuti ketentuan untuk kemudahan menambahkan *field* baru tanpa migrasi

3.4.4. Bukti Penyelesaian

Untuk membuktikan bahwa seluruh *container* dapat di-*deploy* dan *running* secara masing-masing, terdapat *initial state* berupa sembilan *container* yang berjalan dan berstatus healthy. **Gambar 3.4.4.1** menunjukkan Client API yang dihentikan, dibangun ulang tanpa *cache,* dan dijalankan ulang. Selama Client API mati, Aggregator tetap dapat melakukan *polling*. **Gambar 3.4.4.2** menunjukkan Auth Service yang dihentikan, dibangun ulang tanpa *cache,* dan dijalankan ulang. Saat Auth Service mati, token yang diterbitkan masih dapat melayani Client API (HTTP 200) dan login baru gagal (HTTP 000) sesuai dengan yang diharapkan. Diff pada kedua pengujian menunjukkan hanya komponen yang dimatikan yang berubah.

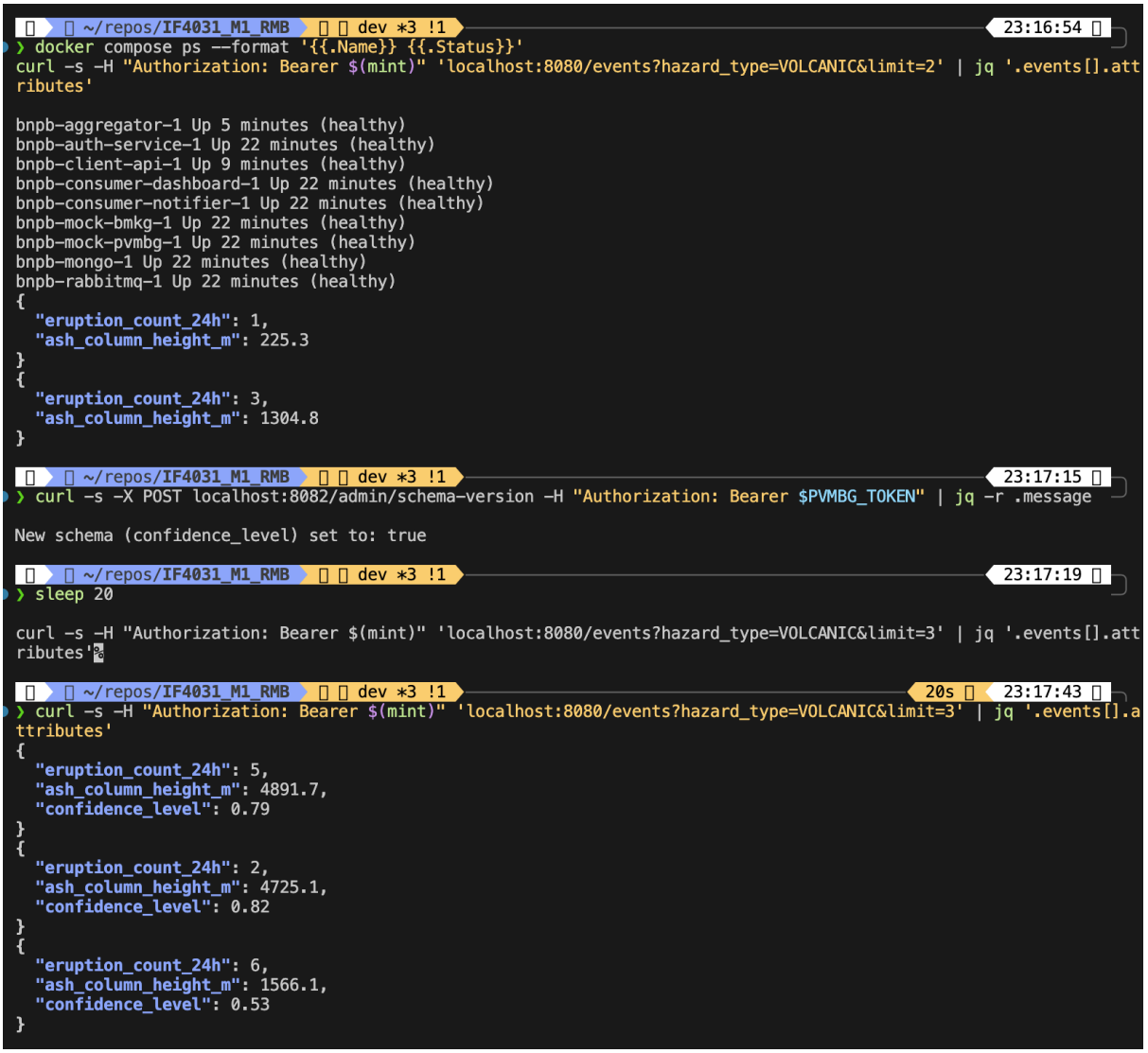


Gambar 3.4.4.1 Bukti Pengujian *Microservice* (Client API)

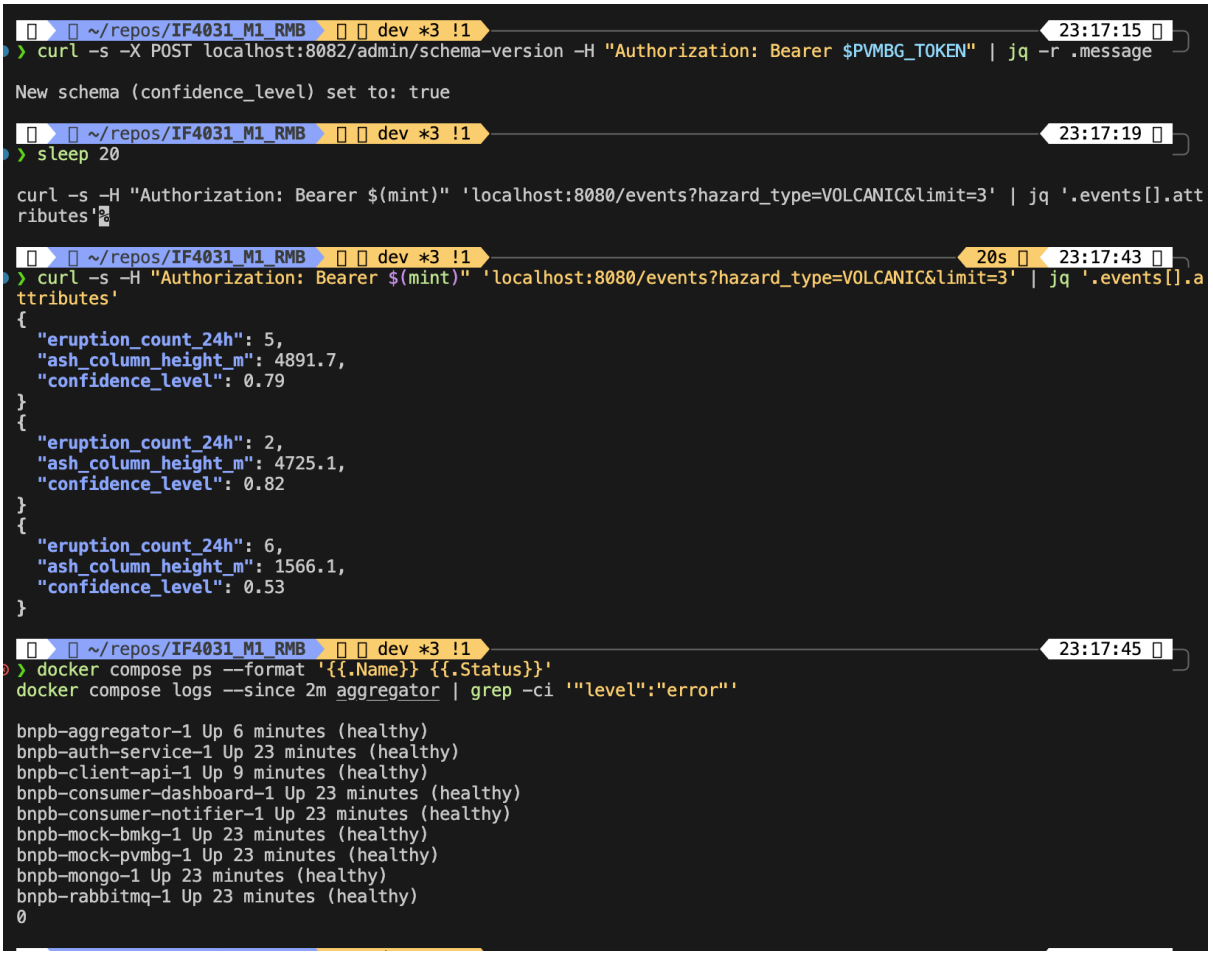


Gambar 3.4.4.2 Bukti Pengujian *Microservice* (Auth Service)

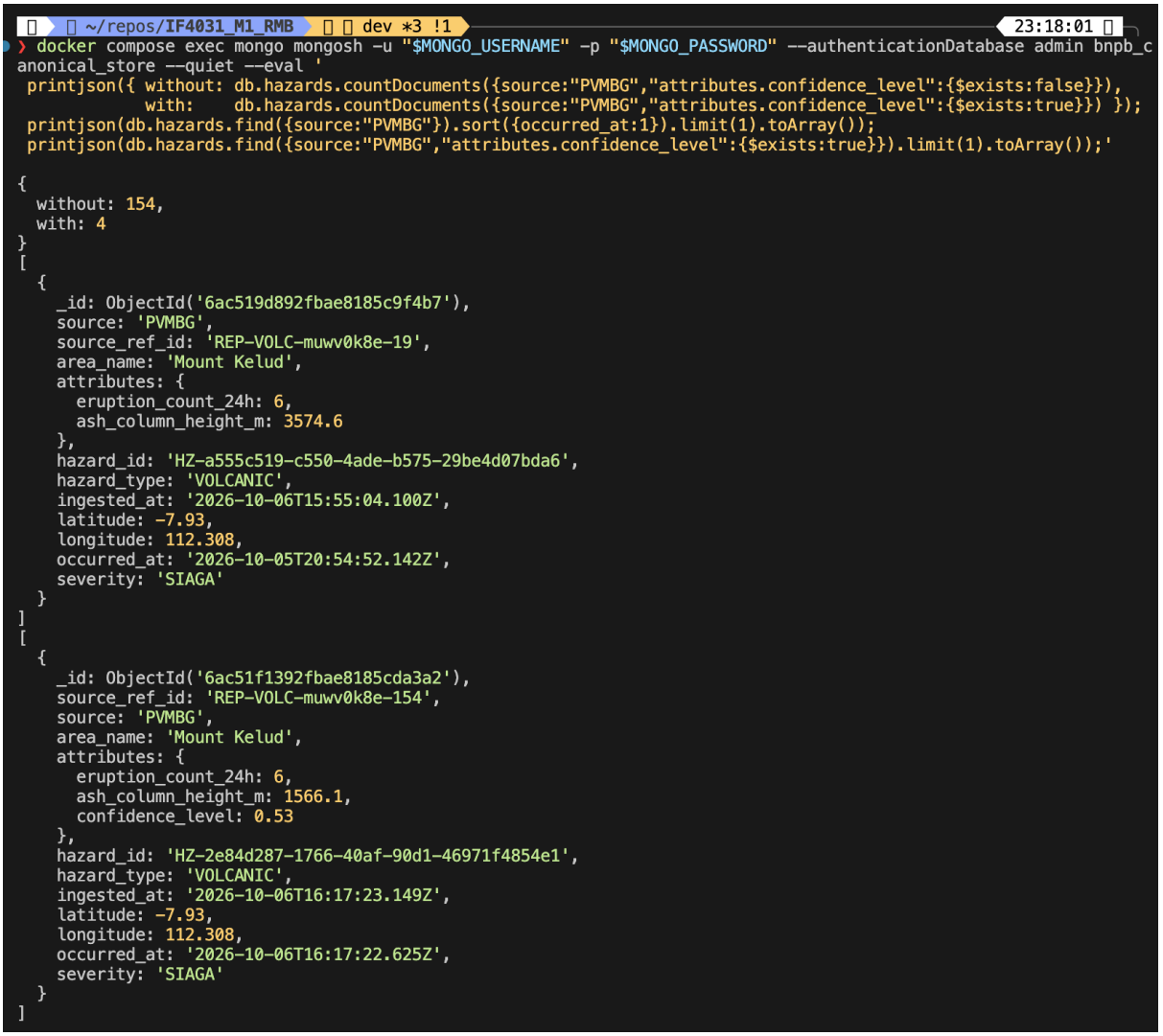
Untuk menguji *flexible storage*, di mana dua bentuk *record* dapat tersimpan secara berdampingan tanpa migrasi, dilakukan pengujian secara langsung dengan menyalakan konfigurasi *confidence level* menjadi *true*. **Gambar 3.4.4.3** menunjukkan skema keadaan sebelum yang tidak memiliki *confidence level* dan keadaan setelah yang menunjukkan API sudah memuat *confidence level*. **Gambar 3.4.4.4** menunjukkan tidak ada *containmer* yang di-*restart*. Gambar 3.4.4.5 menunjukkan kedua bentuk *collection* yang sama, yaitu 154 dokumen PVMBG tanpa *confidence level* dan 4 dengan *confidence level*.



Gambar 3.4.4.3 Bukti Pengujian *Flexible Storage*

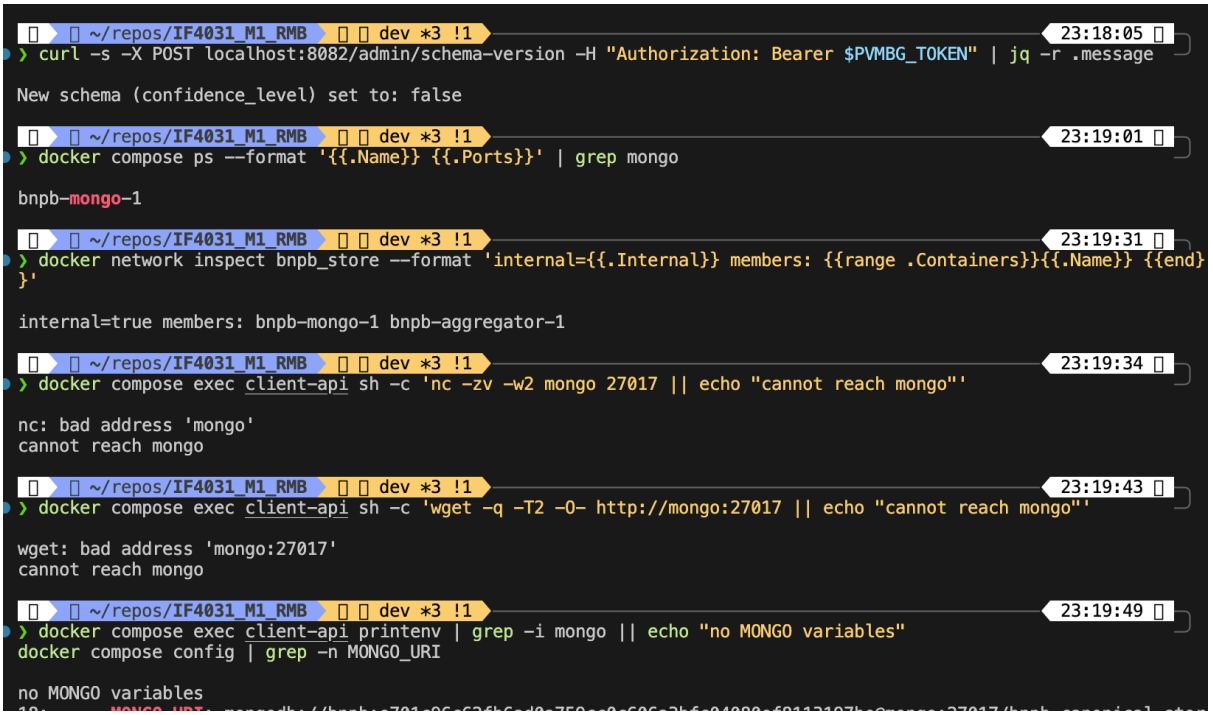


Gambar 3.4.4.4 Bukti Pengujian *Flexible Storage* tanpa *Restart Container*



Gambar 3.4.4.5 Bukti Perbandingan Confidence Level

Terakhir, dilakukan pengujian bahwa hanya Aggregator yang dapat mengakses Canonical Store. **Gambar 3.4.4.6** menunjukkan komponen lain yang gagal ketika mencoba untuk mengakses Canonical Store.



Gambar 3.4.4.6 Bukti Pengujian Hanya Aggregator yang Dapat Mengakses Canonical Store

3.4.5. Pembahasan: Penentuan Batas *Service* dan Alternatif Penyimpanan

Batasan seperti yang dijabarkan pada **Tabel 3.4.1.1**, didasarkan atas: 1) siapa yang boleh mengubah data, 2) siapa yang berubah bersama, dan 3) apa yang harus tetap hidup saat komponen lainnya gagal. Misalnya, Aggregator dan Canonical Store yang dijadikan satu pemilik karena *polling*, *mapping*, dan *write* yang terkait. Terdapat juga Client API, Auth Service, *message broker*, dan *consumer* yang terpisah karena masing-masing dapat berubah dan gagal dengan cara yang berbeda, serta dapat di-*deploy* secara mandiri. Alasan atas keputusan desain juga telah dijabarkan pada **Tabel 3.4.2.1**.

Canonical Store juga dapat diterapkan dengan berbagai alternatif *storage*.

Tabel 3.4.5.1 Alternatif *Storage* untuk Canonical Store

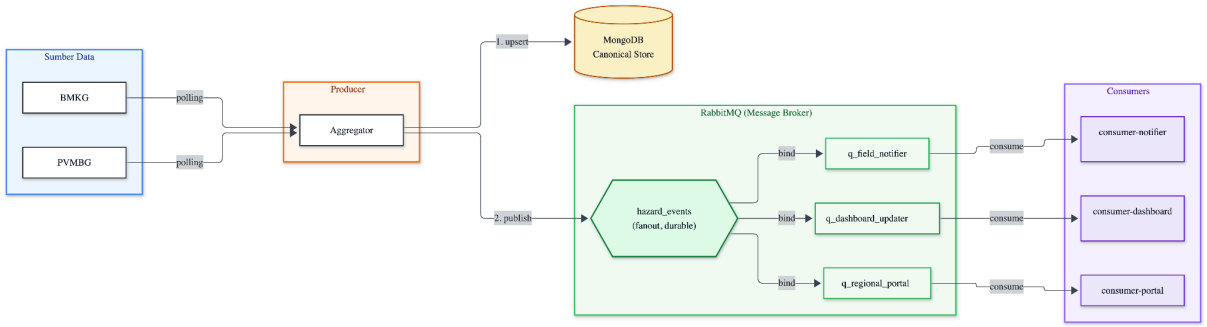
Storage	***Attribute* Dinamis**	**Konsekuensi**
MongoDB	*Supported* secara alami, dengan *fields* yang berbeda-beda untuk masing-masing dokumen	*Field* baru tidak memerlukan migrasi dan indeks dapat digunakan untuk *deduplication*, meskipun transaksi multidokumen lebih lemah dan tidak ada join
PostgreSQL (JSONB)	*Supported*, dengan kolom JSONB	Sangat baik dan efisien untuk *query* multidokumen seperti JOIN, namun *query* isi JSONB lebih rumit
Redis	*Supported*, nilai yang bebas	Tidak mendukung *read* atau *query* berdasarkan *field* atau rentang waktu tanpa indeks tambahan
Relational DB	Tidak	Penambahan *field* baru mengharuskan *migration* dan *deployment*

Berdasarkan perbandingan alternatif penyimpanan di atas, MongoDB dipilih karena merupakan alternatif yang paling alami untuk penyimpanan JSON (dalam format BSON) dengan operasi yang efisien untuk *query* dokumen individual (tanpa skema JOIN), mudah untuk diterapkan *deduplication* dan *idempotency*, dan perubahan skema tidak memaksa *migration*. MongoDB juga mudah untuk di-*scale* secara horizontal dengan menerapkan *sharding*. Kelebihan PostgreSQL yang menjadi alternatif terbaik kedua tidak relevan karena tidak memerlukan transaksi multidokumen.

3.5. *Problem* 5: *Fan-out Event Distribution/Pub-sub*

3.5.1. Rancangan Solusi

Untuk menghindari *bottleneck* dalam pengiriman notifikasi akibat pihak yang lambat, dipilih mekanisme *publish and subscribe* dengan memanfaatkan *message broker* RabbitMQ. Sistem didesain sehingga pemanggilan *consumer* secara langsung tidak dilakukan.



Gambar 3.5.1.1 Diagram Rancangan Solusi *Problem* 5

Setelah data ditarik dari sumber (BMKG/PVMBG), data `HazardEvent` baru disimpan terlebih dahulu oleh aggregator ke dalam MongoDB dengan mekanisme upsert. Setelah data tersimpan, pesan tersebut di-*publish* ke suatu *exchange* RabbitMQ yang dinamakan hazard_events (`index.ts:146-168`). Pengaturan pada *exchange* tersebut dikonfigurasi agar bertipe *fanout* dan berstatus *durable*.

```
async function saveAndPublish(hazards: HazardEvent[], log: Logger,
correlationId: string): Promise<boolean> {
   const inserted: HazardEvent[] = [];
   let allSaved = true;
   const start = performance.now();

   for (const hazard of hazards) {
       try {
           if (await upsertHazard(hazard)) inserted.push(hazard);
       } catch (error) {
           allSaved = false;
           log.error({ source_ref_id: hazard.source_ref_id, error:
errorMessage(error) }, 'Failed to save HazardEvent');
       }
   }
   if (hazards.length > 0) {
       log.info(
           { target: 'mongo', call: 'upsert hazards', count:
hazards.length, inserted: inserted.length, duration_ms:
Math.round(performance.now() - start) },
           'outgoing call ok'
       );
   }

   for (const event of inserted) sendToBroker({ event, correlationId },
log);
   return allSaved;
}
```

Container yang terpisah dialokasikan untuk setiap entitas yang membutuhkan data (*field notifier*, *dashboard updater*, dan *regional portal*). *Queue* dibuat oleh masing-masing layanan tersebut, yang kemudian di-*bind* pada `hazard_events` (`runtime.ts:39-41`).

```
await channel.assertExchange(EXCHANGE, 'fanout', { durable: true });
const q = await channel.assertQueue(queue, { durable: true });
await channel.bindQueue(q.queue, EXCHANGE, '');
```

Dengan rancangan seperti ini, data dari satu sumber dapat diduplikasi dan diteruskan secara asinkron ke antrean masing-masing *consumer* tanpa ada campur tangan tambahan dari sisi *producer*.

3.5.2. Alasan Keputusan

Berikut merupakan alasan masing-masing pengambilan desain beserta alasannya.

Tabel 3.5.2.1 Alasan Pengambilan Keputusan Desain Problem 5

Keputusan Desain	Alasan	Alternatif (Ditolak)
Penggunaan *exchange* bertipe *fanout*.	Desain ini mengurangi ketergantungan kode antar layanan (*coupling*). Pesan dipublikasikan tanpa penentuan tujuan spesifik (*routing key* kosong), sehingga penambahan *consumer* baru dapat ditangani dengan pembuatan antrean baru (bukan memodifikasi kode di sisi aggregator).	Pemanggilan *endpoint* HTTP ke setiap *consumer*, atau penggunaan *Topic Exchange*, memaksa *producer* mengenali identitas pihak penerima.
Alokasi satu *queue* untuk setiap *consumer*.	Desain ini memungkinkan sistem mengurangi bottleneck akibat *consumer* yang lambat. Kegagalan/*delay* pada *consumer* hanya akan memicu penumpukan pesan pada antreannya sendiri, sehingga distribusi pesan ke *consumer* lain tidak terhambat	Penggunaan *shared queue* untuk semua pihak, memungkinkan *bottleneck* akibat satu atau lebih pihak *consumer* yang terhambat.
Konfigurasi *durable* dan pesan *persistent*	Desain ini menjamin antrean dan data yang belum diproses tidak akan hilang meskipun *broker* atau *consumer* mengalami *restart*.	Konfigurasi menggunakan memori pada *broker*, memungkinkan terjadinya kehilangan data saat terjadi kegagalan kontainer.

3.5.3. Batasan

Berikut merupakan daftar batasan yang masih terdapat pada sistem dengan desain terpilih.

Tabel 3.5.3.1 Batasan Desain Problem 5

Batasan	Deskripsi dan Alasan
Ketiadaan *transactional outbox*	Penyimpanan data dan publikasi ke *broker* masih dieksekusi sebagai dua tahapan yang berurutan namun terpisah. Jika Aggregator mengalami kegagalan (*crash*) tepat di antara kedua tahapan tersebut, event akan tersimpan secara permanen namun tidak terpublikasi ke *consumer*.
Tidak ada *queue* untuk pesan gagal	Antrean khusus untuk menampung pesan gagal tidak diimplementasikan. Pesan yang gagal diproses oleh *consumer* akan ditolak tanpa mekanisme pengembalian ke antrian.

3.5.4. Bukti Penyelesaian

Untuk membuktikan bahwa desain sudah diimplementasikan, pengujian pada arsitektur *publish*/*subscribe* dijalankan melalui skenario yang dianggap sesuai dengan kondisi nyata sistem. Pengujian ini mencakup validasi topologi, simulasi kegagalan komponen, dan penambahan layanan baru.

```
-- exchange
name    type    durable
hazard_events   fanout  true
-- bindings (exchange -> queue)
source_name     destination_name
hazard_events   q_dashboard_updater
hazard_events   q_field_notifier
-- queues
name    durable messages_ready  messages_unacknowledged messages_persistent     consumers
q_field_notifier        true    0       0       0       1
q_dashboard_updater     true    0       0       0       1
```

Gambar 3.5.4.1 Konfigurasi *Exchange Fanout* dan *Binding* Antrean pada RabbitMQ

Dapat dilihat pada **Gambar 3.5.4.1**, *exchange* hazard_eventssudah dikonfigurasi dengan tipe *fanout* dan berstatus *durable*. Masing-masing antrean milik *consumer* juga sudah di-*bind* ke *exchange* tersebut.

```
-- references to consumers or queues inside aggregator/src (expected: 0)
0
-- aggregator dependencies in docker-compose.yml (broker and store only, no consumer)
    depends_on:
      mongo:
        condition: service_healthy
      rabbitmq:
        condition: service_healthy
    healthcheck:
```

Gambar 3.5.4.2 Tidak Ada *Reference to Consumers* pada *Source Code* dan Konfigurasi *Aggregator Dependencies*

Terlihat pada **Gambar 3.5.4.2**, tidak ditemukan satu pun referensi kode *consumer* di dalam kode sumber Aggregator, sehingga dapat dipastikan bahwa tidak ada *coupling* di sini.


```
-- recent events published by the aggregator
2026-10-09T05:06:36.174Z aggregator hazard_id=HZ-c9d3a757-f35b-4753-9efb-87afa3fcd430 source=PVMBG severity=WASPADA Published HazardEvent
2026-10-09T05:06:40.132Z aggregator hazard_id=HZ-0cb59976-4306-439e-a5d2-aa594b845f45 source=BMKG severity=WASPADA Published HazardEvent
2026-10-09T05:06:48.114Z aggregator hazard_id=HZ-bd2f0039-4534-4734-9676-ed6f5f36d3c0 source=PVMBG severity=WASPADA Published HazardEvent
2026-10-09T05:06:52.158Z aggregator hazard_id=HZ-a1d2e92e-ca0c-4e5c-8cbb-ac429fd7b170 source=BMKG severity=WASPADA Published HazardEvent
2026-10-09T05:06:55.936Z aggregator hazard_id=HZ-b1287de2-6617-4832-8146-bae02731e828 source=PVMBG severity=AWAS Published HazardEvent
-- one message as stored in the broker (temporary queue q_inspect, waits 15s)
{"delivery_mode":2,"message_id":"HZ-7076a7e9-a91a-449d-8537-23463db9caec:NORMAL","correlation_id":"poll-b7bf27d2-29a5-4dc7-9692-91a4719dcd25","content_type":"application/json","payload":{"hazard_id":"HZ-7076a7e9-a91a-449d-8537-23463db9caec","source":"BMKG","source_ref_id":"EV-BMKG-muz32fcz-5452","hazard_type":"SEISMIC","severity":"NORMAL","area_name":"Coastal Region 5452","latitude":-7.520934418714854,"longitude":110.99056157533975,"occurred_at":"2026-10-09T05:06:59.203Z","ingested_at":"2026-10-09T05:07:01.169Z","attributes":{"magnitude":4.7,"depth_km":48.7,"potential_tsunami":false}}}
```


Gambar 3.5.4.3 Publikasi *Event* oleh Aggregator dan Verifikasi Penyimpanan Pesan Berstatus *Persistent* pada Broker

Saat operasi berjalan, aliran data dapat dilihat (secara asinkron). Sesuai **Gambar 3.5.4.3**, pesan *event* berhasil di-*publish* oleh Aggregator dan tersimpan di *broker* dengan mode *persistent* (`delivery_mode=2`).

```
-- consumer containers
aggregator Up 24 hours (healthy)
consumer-dashboard Up 24 hours (healthy)
consumer-notifier Up 11 minutes (healthy)
-- consumers attached to the broker (one channel per queue)
queue_name      channel_pid     ack_required    prefetch_count
q_field_notifier        <rabbit@d55d63d916e8.1791521789.1027.0> true    10
q_dashboard_updater     <rabbit@d55d63d916e8.1791521789.997.0>  true    10
```

Gambar 3.5.4.4 Status *Container* dan *Queue* Aktif untuk Dua *Consumer*

Dapat dilihat pada **Gambar 3.5.4.4** bahwa dua *consumer* dengan *queue* aktif untuk masing-masing *consumer* tersebut sudah dikonfigurasi.

```
-- one hazard event seen by the producer and by both consumers: HZ-a71ad5b8-c382-4624-9f1b-981e6168ca7b
2026-10-09T05:08:22.211Z aggregator correlation_id=poll-105381af-b3bc-44b9-a694-b2b42acca79a hazard_id=HZ-a71ad5b8-c382-4624-9f1b-981e6168ca7b severity=WASPADA Published HazardEvent
2026-10-09T05:08:22.210Z consumer-dashboard correlation_id=poll-105381af-b3bc-44b9-a694-b2b42acca79a hazard_id=HZ-a71ad5b8-c382-4624-9f1b-981e6168ca7b severity=WASPADA Received update for UI processing
2026-10-09T05:08:22.210Z consumer-notifier correlation_id=poll-105381af-b3bc-44b9-a694-b2b42acca79a hazard_id=HZ-a71ad5b8-c382-4624-9f1b-981e6168ca7b severity=WASPADA Event ignored
```

Gambar 3.5.4.5 Penerimaan *Event* Secara Serentak oleh Dua *Consumer*

Setelah kedua *consumer* tersebut dikonfigurasi, diperlihatkan pada **Gambar 3.5.4.5** bahwa suatu *event* peringatan berhasil didistribusikan ke kedua *consumer* secara serentak oleh `consumer-dashboard` dan `consumer-notifier`.

```
-- baseline
name    durable messages_ready  messages_unacknowledged messages_persistent     consumers
q_field_notifier        true    0       0       0       1
q_dashboard_updater     true    0       0       0       1
```

Gambar 3.5.4.6 Kondisi Antrean tanpa Tumpukan Pesan

Setelah sistem dipastikan berjalan normal (dengan *queue* yang kosong) sesuai **Gambar 3.5.4.6**, dilakukan simulasi kegagalan di mana layanan `consumer-notifier` dimatikan.

```
[+] Stopping 1/1
 ✔ Container bnpb-consumer-notifier-1  Stopped                                                                    0.2s
T_STOP=2026-10-09T05:09:29Z
-- queues while consumer-notifier is down
name    durable messages_ready  messages_unacknowledged messages_persistent     consumers
q_field_notifier        true    9       0       9       0
q_dashboard_updater     true    0       0       0       1
```

Gambar 3.5.4.7 `consumer-notifier` Dimatikan

Dengan mematikan consumer-notifier pada **Gambar 3.5.4.7**, tumpukan pesan terjadi pada `q_field_notifier`. Dapat dilihat bahwa `q_dashboard_updater` tetap beroperasi.

```
-- publish times during the outage (producer was not blocked)
05:09:31.218 05:09:36.464 05:09:40.214 05:09:46.716 05:09:49.255 05:09:56.603 05:10:01.227 05:10:10.092 05:10:10.226
published by aggregator during outage : 9
received by consumer-dashboard        : 18
processed by consumer-notifier        : 0
published but not received by consumer-dashboard (expected: none):
-- aggregator after the outage window
{"status":"ok","service":"aggregator","store_connected":true,"broker_connected":true}
aggregator cid=f5621f69a064 img=sha256:2eefb54fc2ce started=2026-10-08T05:16:09.971926438Z restarts=0
```

Gambar 3.5.4.8 Operasi Dilanjutkan selama `consumer-notifier` Mati

Gambar di atas menunjukkan bahwa selama simulasi berlangsung, Aggregator tidak mengalami restart dan terus melakukan *supply* pesan ke *consumer* yang aktif tanpa ter-blok.

```
T_START=2026-10-09T05:11:27Z
[+] Running 2/2
 ✔ Container bnpb-rabbitmq-1           Healthy                                                                    0.5s
 ✔ Container bnpb-consumer-notifier-1  Started                                                                    0.2s
-- queues after consumer-notifier is back
name    durable messages_ready  messages_unacknowledged messages_persistent     consumers
q_field_notifier        true    0       0       0       1
q_dashboard_updater     true    0       0       0       1
```

Gambar 3.5.4.9 Pengosongan *Backlog* Pesan

Ketika consumer-notifier dihidupkan kembali, pemulihan pesan langsung dilakukan. Sesuai **Gambar 3.5.4.9**, *backlog* pada *queue* langsung habis (karena sudah diproses).

```
published while consumer-notifier was down     : 24
processed by consumer-notifier after restart   : 46 (includes events published after restart)
published during the outage but never processed (expected: none):
-- per event: time published by aggregator, time processed by consumer-notifier
HZ-04fe6f81-6145-407a-bc65-6317bc39d02c:NORMAL published=05:09:31.218 processed=05:11:28.822
HZ-47ba38eb-182a-435c-9d89-f411679c6a4e:NORMAL published=05:09:36.464 processed=05:11:28.822
HZ-8f23ecc7-04d2-4d12-bae3-327e8a2b0c4d:NORMAL published=05:09:40.214 processed=05:11:28.823
HZ-467816ab-ada2-4725-85bf-68f18f202646:NORMAL published=05:09:46.716 processed=05:11:28.824
HZ-d01c8489-0212-4649-97cf-57fbd906e17a:NORMAL published=05:09:49.255 processed=05:11:28.824
HZ-53a7abe8-9dd9-4d0c-81ef-1ddc646a10cd:WASPADA published=05:09:56.603 processed=05:11:28.825
HZ-29e41c15-0b63-4d5d-943a-b7f57ffca322:SIAGA published=05:10:01.227 processed=05:11:28.825
HZ-bfafe7c7-769e-429b-83b9-2c19763f4646:NORMAL published=05:10:10.092 processed=05:11:28.825
HZ-c339f46f-a31e-482c-ac37-d035110e40f7:NORMAL published=05:10:10.226 processed=05:11:28.826
HZ-f3604ad0-a050-4e9c-88e8-572c2c3bc995:NORMAL published=05:10:17.911 processed=05:11:28.827
HZ-62a83f69-9a7c-475a-9ed9-5d29bac51678:WASPADA published=05:10:22.236 processed=05:11:28.829
HZ-ca52477d-f0d1-45eb-a2be-24dfa19acb7d:WASPADA published=05:10:26.399 processed=05:11:28.829
HZ-aa7b1160-eb1d-42ba-9a19-2c3154aaa159:WASPADA published=05:10:31.251 processed=05:11:28.829
HZ-ab777faa-d683-4f4f-8957-8c9e5004f7c3:SIAGA published=05:10:36.966 processed=05:11:28.830
HZ-4305aacf-be9c-4846-8cee-8cf290023be9:NORMAL published=05:10:40.269 processed=05:11:28.831
HZ-052e12a8-fda6-4acb-85ff-8a2db86d5561:NORMAL published=05:10:45.751 processed=05:11:28.832
HZ-b66469f6-84d7-4d7d-ba24-fa48daf9c6aa:NORMAL published=05:10:49.283 processed=05:11:28.833
HZ-85525084-ed73-4430-8ef4-552f7ad8a875:SIAGA published=05:10:55.887 processed=05:11:28.834
HZ-1f3254b5-d52a-4584-973c-80ded9bec941:WASPADA published=05:11:01.290 processed=05:11:28.834
HZ-c01fa2a6-5f87-49a1-81a2-7654f80c6992:NORMAL published=05:11:07.127 processed=05:11:28.835
HZ-28c1ece0-4561-497f-b64a-a3d6f661b2de:WASPADA published=05:11:10.247 processed=05:11:28.836
HZ-b1ac99c0-ae64-4617-8f3c-cdf0100cab94:WASPADA published=05:11:16.140 processed=05:11:28.836
HZ-f99bf5d3-74f3-4764-aa55-1432cf3f36fe:WASPADA published=05:11:19.295 processed=05:11:28.837
HZ-2420c620-614d-4de8-811a-5bb98597f134:WASPADA published=05:11:26.305 processed=05:11:28.837
```

Gambar 3.5.4.10 Kelengkapan Pemrosesan Event Setelah Konsumen Dihidupkan Kembali

Pada **Gambar 3.5.4.10**, seluruh pesan yang sempat tertunda berhasil diproses tanpa ada satupun data yang hilang. Hal ini dapat terjadi karena antrean dikonfigurasi menjadi *durable* dan wajib ACK manual.

```
-- queue settings that explain the catch-up
name    durable auto_delete     exclusive       arguments
q_field_notifier        true    false   false   [{"x-queue-type","classic"}]
q_dashboard_updater     true    false   false   [{"x-queue-type","classic"}]
policies:

consumers (ack_required=true means manual ack):
queue_name      channel_pid     ack_required    prefetch_count
q_field_notifier        <rabbit@d55d63d916e8.1791521789.1417.0> true    10
q_dashboard_updater     <rabbit@d55d63d916e8.1791521789.997.0>  true    10
```

Gambar 3.5.4.11 Konfigurasi Antrean *Durable* dan Konfirmasi ACK Manual

```
== Step 3b: backlog survives a broker restart (criterion 3, durability)
[+] Stopping 1/1
 ✔ Container bnpb-consumer-notifier-1  Stopped                                                                    0.2s
-- queues before broker restart
name    durable messages_ready  messages_unacknowledged messages_persistent     consumers
q_field_notifier        true    5       0       5       0
q_dashboard_updater     true    0       0       0       1
[+] Restarting 1/1
 ✔ Container bnpb-rabbitmq-1  Started                                                                            1.5s
-- queues after broker restart (messages_ready must not be lower than before)
name    durable messages_ready  messages_unacknowledged messages_persistent     consumers
q_field_notifier        true    9       0       9       0
q_dashboard_updater     true    0       0       0       1
[+] Running 2/2
 ✔ Container bnpb-rabbitmq-1          Healthy                                                                     0.5s
 ✔ Container bnpb-consumer-notifier-1  Started                                                                    0.3s
-- queues after consumer-notifier is back
name    durable messages_ready  messages_unacknowledged messages_persistent     consumers
q_field_notifier        true    0       0       0       1
q_dashboard_updater     true    0       0       0       1
```

Gambar 3.5.4.12 *Backlog* Tahan terhadap Restart pada RabbitMQ

Dapat dilihat pada **Gambar 3.5.4.12** bahwa *backlog* yang belum sempat diproses tidak hilang meskipun RabbitMQ sudah mengalami *restart*.

```
-- before
name    durable messages_ready  messages_unacknowledged messages_persistent     consumers
q_field_notifier        true    0       0       0       1
q_dashboard_updater     true    0       0       0       1
source_name     destination_name
hazard_events   q_dashboard_updater
hazard_events   q_field_notifier
aggregator cid=f5621f69a064 img=sha256:2eefb54fc2ce started=2026-10-08T05:16:09.971926438Z restarts=0
aggregator source checksum: e118689a62e232a6
```

Gambar 3.5.4.13 Identitas dan *Checksum* sebelum Penambahan *Consumer*

Sekarang, dilakukan pengujian skalabilitas dengan melakukan penambahan *consumer*. Pada **Gambar 3.5.4.13**, dapat dilihat status awal Aggregator sebelum dilakukan penambahan *consumer*.

```
-- after
name    durable messages_ready  messages_unacknowledged messages_persistent     consumers
q_field_notifier        true    0       0       0       1
q_regional_portal       true    0       0       0       1
q_dashboard_updater     true    0       0       0       1
source_name     destination_name
hazard_events   q_dashboard_updater
hazard_events   q_field_notifier
hazard_events   q_regional_portal
aggregator cid=f5621f69a064 img=sha256:2eefb54fc2ce started=2026-10-08T05:16:09.971926438Z restarts=0
aggregator source checksum: e118689a62e232a6
aggregator unchanged (same container, same start time, same source): yes
```

Gambar 3.5.4.14 Status Aggregator Konsisten

Dapat dilihat pada **Gambar 3.5.4.14**, bahwa proses *scaling* yang diuji berhasil. *Queue* baru yang ditambahkan tidak mengubah *container ID*, waktu mulai, jumlah *restart*, maupun *checksum* Aggregator. Layanan juga terbukti siap menerima *event* sesuai dengan **Gambar 3.5.4.15**.

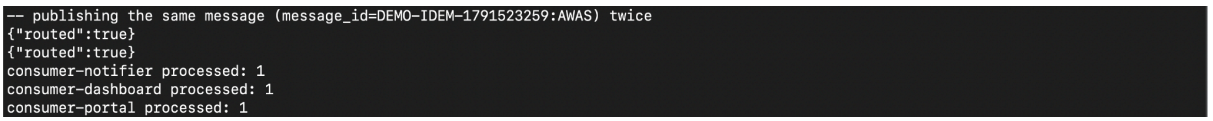
```
-- one hazard event seen by the producer and all three consumers: HZ-4fc63cb0-4f3c-4825-a8a8-42cf1c7b505d
2026-10-09T05:19:40.466Z aggregator correlation_id=poll-ff71bc6e-6cf7-49df-829d-10f659a40058 hazard_id=HZ-4fc63cb0-4f3c-4825-a8a8-42cf1c7b5
05d severity=SIAGA Published HazardEvent
2026-10-09T05:19:40.465Z consumer-dashboard correlation_id=poll-ff71bc6e-6cf7-49df-829d-10f659a40058 hazard_id=HZ-4fc63cb0-4f3c-4825-a8a8-4
2cf1c7b505d severity=SIAGA Received update for UI processing
2026-10-09T05:19:40.464Z consumer-notifier correlation_id=poll-ff71bc6e-6cf7-49df-829d-10f659a40058 hazard_id=HZ-4fc63cb0-4f3c-4825-a8a8-42
cf1c7b505d severity=SIAGA SENDING EMERGENCY ALERT
2026-10-09T05:19:40.464Z consumer-portal correlation_id=poll-ff71bc6e-6cf7-49df-829d-10f659a40058 hazard_id=HZ-4fc63cb0-4f3c-4825-a8a8-42cf
1c7b505d severity=SIAGA Regional portal feed updated
```

Gambar 3.5.4.15 *Event* Berhasil Didistribusikan tanpa Modifikasi *Producer*

```
-- optional: a fourth consumer with no code (queue and binding created through the management API)
source_name     destination_name
hazard_events   q_dashboard_updater
hazard_events   q_field_notifier
hazard_events   q_regional_portal
hazard_events   q_extra_consumer
{"hazard_id":"HZ-ef6c3095-ca5e-405d-ad5d-eb04c01ac17d","severity":"AWAS"}
{"hazard_id":"HZ-d44cc92b-4b0f-4ae3-b103-0d6c0f09df34","severity":"WASPADA"}
```

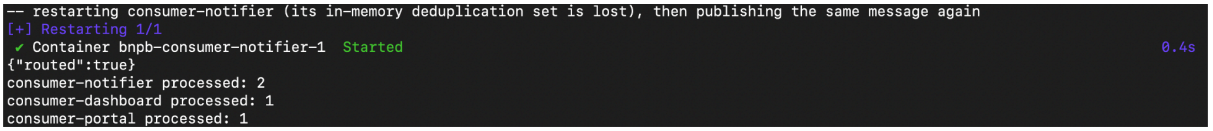
Gambar 3.5.4.16 Penambahan *Queue* Langsung melalui *Broker Management* API

Pada **Gambar 3.5.4.16**, dibuktikan bahwa *queue* baru tersebut sukses menerima data secara otomatis tanpa memerlukan kompilasi ulang di sisi Aggregator.



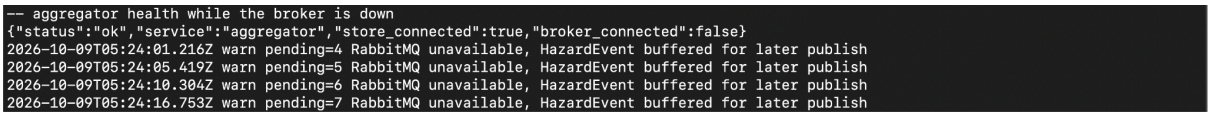
Gambar 3.5.4.17 Mekanisme Deduplikasi *In-Memory*

Pada kenyataannya, akan terdapat pesan yang bersifat *duplicate*. Ketika dua pesan identik disimulasikan masuk ke dalam sistem, terbukti bahwa mekanisme deduplikasi *in-memory* bekerja sesuai fungsinya. Dapat dilihat pada **Gambar 3.5.4.17**, setiap *consumer* hanya memproses event satu kali.



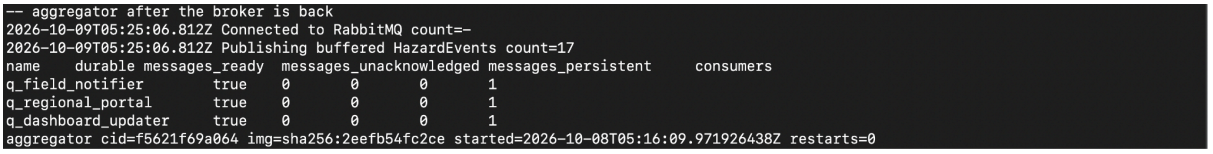
Gambar 3.5.4.18 Pemrosesan Ulang Pesan Ganda Setelah *Restart* Konsumen (*At-Least-Once*)

Pada **Gambar 3.5.4.18**, dapat diamati bahwa ketika *consumer* di-*restart* (memori terhapus), pesan *duplicated* yang masuk kembali akan diproses ulang, artinya jaminan pengiriman sistem ini *at-least-once*.



Gambar 3.5.4.19 Pesan Dialihkan ke *Memory* Aggregator saat Broker Mati

Di sini, diuji skenario di mana terjadi kegagalan pada *broker* (RabbitMQ). Sesuai pada **Gambar 3.5.4.19**, Aggregator tidak mengalami *crash*. Pesan-pesan yang ditarik (tapi gagal di-*publish*) dipindahkan ke dalam *in-memory buffer*.



Gambar 3.5.4.20 Aggregator Tidak Perlu Restart untuk Publikasi setelah *Broker* Mati

Setelah RabbitMQ kembali pulih, sesuai **Gambar 3.5.4.20**, Aggregator langsung tersambung ulang dan seluruh pesan yang tertahan di *buffer* langsung di-*publish* tanpa memerlukan *restart* pada sisi Aggregator.

Tabel Kontribusi

NIM	Nama	Kontribusi
13523013	Nathaniel Jonathan Rusli	Canonical Store, Client API, *concurrency, graceful degradation*, *load testing*
13523015	Maheswara Bayu Kaindra	Mock BMKG & PVMBG, Aggregator, *data mapping*, *fault tolerance consumer*
13523037	Buege Mahara Putra	Auth Service, *scope filtering*, Docker *orchestration*

Lampiran

a. Repository : https://github.com/MaheswaraKaindra/IF4031_RMB

b. Form Penggunaan AI Berdasarkan Panduan AI ITB

Dengan ini, saya/kami* :
Nama/NIM : Nathaniel Jonathan Rusli, Maheswara Bayu Kaindra, Buege Mahara Putra / 13523013, 13523015, 13523037
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah : IF4031
Nama Mata Kuliah : Arsitektur Aplikasi Terdistribusi
Judul Tugas/Proposal : Tugas Besar IF4031 Arsitektur Aplikasi Terdistribusi Milestone 1

Menyatakan bahwa ~~saya~~/**kami*** **menggunakan**~~/tidak menggunakan~~* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/**Tidak***	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	~~Ya~~/**Tidak***	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	~~Ya~~/**Tidak***	
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/**Tidak***	
Penulisan Kode Program: menggunakan AI *coding agent* seperti Claude Code.	**Ya**~~/Tidak~~*	*AI task completion-human evaluation*

~~Saya~~/**Kami*** juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 10 Oktober 2026



Nathaniel Jonathan Rusli
NIM 13523013



Mahesawara Bayu Kaindra
NIM 13523015



Buege Mahara Putra
NIM 13523037